

SAPIENZA
UNIVERSITÀ DI ROMA

Fine-Tuning Large Language Models via Peer-Generated Training Data

Faculty of Mathematical, Physical and Natural Sciences
Bachelor's Degree in Mathematical Sciences for Artificial Intelligence

Lorenzo Salis
ID number 2130242

Advisor
Prof. Fabrizio Silvestri

Academic Year 2025/2026

Fine-Tuning Large Language Models via Peer-Generated Training Data
Sapienza University of Rome

© 2026 Lorenzo Salis. All rights reserved

This thesis has been typeset by LaTeX and the Sapthesis class.

Author's email: salis.2130242@studenti.uniroma1.it

Abstract

Large Language Models (LLMs) have by now taken on a leading role in natural language processing, making their ability to produce responses aligned with user expectations increasingly relevant. This *alignment* problem is typically addressed through Reinforcement Learning from Human Feedback (RLHF), whose cost and limited scalability motivate the search for automatic alternatives to the collection of human annotations.

This work assesses the feasibility of fine-tuning via Direct Preference Optimization (DPO) starting from preference datasets generated automatically by the mutual evaluation framework AutoBench, without any human annotation. Starting from the collective scores produced by AutoBench, preference datasets were built according to two extraction strategies (Best-vs-Worst and Pairwise), and used for the fine-tuning of two small open-source models, Gemma 2 2B Instruct and Qwen 2.5 0.5B Instruct, in a limited computational environment.

The fine-tuned models were evaluated on an internal test set derived from AutoBench and on standardized public benchmarks. The results show that the approach is technically feasible and preserves the models' general capabilities, and that the quality of the preference pairs affects final performance more than their quantity. For Qwen 2.5 0.5B, a length bias is also observed in the log-probabilities used to compute the preference accuracy, which systematically penalizes its evaluation even after normalization; the residual phenomenon is discussed in light of a *typicality bias* hypothesis.

Contents

1 Introduction **1**

2 Theoretical Background **3**
2.1 Large Language Models . . . 3
2.1.1 The Transformer Architecture . . . 3
2.1.2 Pre-training . . . 4
2.1.3 Instruction Tuning . . . 5
2.1.4 Limitations of Pre-trained Models . . . 5
2.2 Fine-Tuning of Large Language Models . . . 5
2.2.1 Full Fine-Tuning . . . 6
2.2.2 Parameter-Efficient Fine-Tuning . . . 6
2.2.3 Low-Rank Adaptation (LoRA) . . . 6
2.2.4 QLoRA . . . 7
2.3 Preference-Based Alignment . . . 8
2.3.1 Reinforcement Learning from Human Feedback . . . 8
2.3.2 The Reward Model . . . 8
2.3.3 Proximal Policy Optimization (PPO) . . . 9
2.3.4 Limitations of RLHF . . . 10
2.4 Direct Preference Optimization . . . 11
2.4.1 The Idea Behind DPO . . . 11
2.4.2 Dataset Structure: Prompt, Chosen and Rejected . . . 12
2.4.3 Advantages over RLHF . . . 12
2.4.4 Why DPO Is Suitable for This Work . . . 12
2.5 Peer-Generated Training Data . . . 13
2.5.1 LLM-as-a-Judge . . . 13
2.5.2 Peer Assessment . . . 13
2.5.3 Automatic Preference Datasets . . . 14
2.5.4 Advantages and Limitations . . . 14
2.5.5 Introduction to AutoBench . . . 14

3 AutoBench and the Construction of Preference Datasets **16**
3.1 AutoBench as a Source of Peer-Generated Data . . . 16
3.2 Data Preparation and Cleaning . . . 17
3.3 Extraction of Preference Pairs . . . 19
3.3.1 Best-vs-Worst Strategy . . . 19
3.3.2 Pairwise Strategy . . . 20

3.4 Quality Filters . . . 21
3.4.1 Preference Gap Filter . . . 21
3.4.2 Textual Similarity Filter . . . 21
3.4.3 Limiting the Number of Pairs per Prompt . . . 22
3.5 Construction of the Final Datasets . . . 22
3.5.1 Best-vs-Worst Dataset . . . 23
3.5.2 Pairwise Dataset . . . 23
3.5.3 Comparison Between the Datasets . . . 24
3.6 Data Splitting . . . 24

4 Experimental Setup **27**
4.1 Experimental Objectives . . . 27
4.2 Experimental Environment . . . 27
4.3 Fine-Tuning Procedure . . . 28
4.3.1 Quantization and QLoRA . . . 28
4.3.2 Training Configuration . . . 29
4.3.3 Models Considered . . . 30
4.4 Experiment Configuration . . . 30
4.5 Evaluation Methodology . . . 31
4.5.1 Evaluation on the AutoBench Test Set . . . 32
4.5.2 Raw Accuracy and Normalized Accuracy . . . 32
4.5.3 External Benchmarks . . . 32
4.5.4 Evaluation Framework . . . 33

5 Experimental Results **34**
5.1 Introduction to the Results . . . 34
5.2 Gemma 2 2B + Best-Worst Dataset . . . 34
5.2.1 Results on the AutoBench Test Set . . . 35
5.2.2 Results on the External Benchmarks . . . 35
5.3 Qwen 2.5 0.5B + Best-Worst Dataset . . . 36
5.3.1 Results on the AutoBench Test Set . . . 36
5.3.2 Results on the External Benchmarks . . . 36
5.4 Qwen 2.5 0.5B + Pairwise Dataset ($\Delta = 0.3$) . . . 37
5.4.1 Results on the AutoBench Test Set . . . 37
5.4.2 Results on the External Benchmarks . . . 38
5.5 Qwen 2.5 0.5B + Pairwise Dataset ($\Delta = 0.5$) . . . 38
5.5.1 Results on the AutoBench Test Set . . . 39
5.5.2 Results on the External Benchmarks . . . 39
5.6 Summary of the Results . . . 40

6 Discussion of the Results **41**
6.1 Assessing the Effectiveness of Peer-Generated Training Data . . . 41
6.2 Influence of Preference Dataset Quality . . . 42
6.3 Critical Interpretation of the Experimental Results . . . 43
6.4 Biases Observed During Evaluation . . . 44
6.4.1 Length Bias . . . 44
6.4.2 An Additional Hypothesis: Typicality Bias . . . 44

6.5 Limitations of This Work 46
6.6 Future Directions 47

7 Conclusions **49**

Chapter 1

Introduction

In recent years, *Large Language Models* (LLMs) have taken on a central role in automatic natural language processing, demonstrating remarkable capabilities across a wide variety of language tasks. Alongside progress in architectures and training procedures, a fundamental challenge concerns *alignment*: the ability to steer the behavior of these models toward responses consistent with the preferences and expectations of users. The most established paradigm for addressing this problem, Reinforcement Learning from Human Feedback (RLHF), relies on the collection of judgments expressed by human annotators, a process that guarantees high quality but is costly, slow and hard to scale.

This limitation has motivated growing interest in automatic methods for building preference data, in which human judgment is replaced by that of other language models. This work falls within this line of research, exploring the feasibility of an approach based entirely on *peer-generated training data*: preference data derived from the mutual evaluation among language models, without resorting to human annotations.

To this end, we use AutoBench, an automatic evaluation framework in which the participating models simultaneously take on the role of generators and evaluators, producing scores that reflect a collective judgment on the quality of the responses. The data thus obtained are transformed into preference datasets in the format required by Direct Preference Optimization (DPO), an alignment algorithm that allows the parameters of a language model to be updated directly from comparisons between responses, avoiding the need to train a separate reward model. The experiments are conducted on two small open-source models, Gemma 2 2B Instruct and Qwen 2.5 0.5B Instruct, in a deliberately limited computational environment, in order to verify the practicality of the approach even outside high-resource settings.

The main contributions of this work are the following:

- the design and implementation of a complete pipeline for transforming raw AutoBench data into preference datasets compatible with DPO training, including filtering and deduplication procedures;
- the construction and comparison of several preference extraction strategies (Best-vs-Worst and Pairwise with different thresholds), with an analysis of the impact of the quality of the preference signal on final performance;

- a systematic experimental evaluation of the fine-tuned models both on an internal test set derived from AutoBench and on standardized public benchmarks;
- the observation and analysis of the effect of the length bias of unnormalized log-probabilities on the preference accuracy observed for Qwen 2.5 0.5B, highlighting some methodological implications for automatic evaluation using preference datasets.

Thesis Structure

Chapter 2 introduces the theoretical concepts needed to understand the work: the Transformer architecture, efficient fine-tuning techniques (LoRA and QLoRA), the RLHF paradigm and its limitations, the DPO algorithm and the main approaches to the automatic construction of preference data. Chapter 3 describes AutoBench as a source of *peer-generated* data and illustrates the pipeline for building the preference datasets, from cleaning and normalization operations through to the extraction strategies and quality filters adopted. Chapter 4 presents the experimental setup: the models used, the computational environment, the training hyperparameters and the evaluation methodology. Chapter 5 presents the results obtained in the various experiments, analyzing the performance of the models on the different preference datasets. Chapter 6 discusses the results in the broader context of the literature, highlights their limitations and proposes possible directions for future research. Chapter 7 concludes the work with a summary of the contributions and the main observations.

Chapter 2

Theoretical Background

2.1 Large Language Models

Natural Language Processing (NLP) has been profoundly transformed by the rise of Large Language Models (LLMs), which have achieved unprecedented performance across a wide variety of language tasks. Models such as GPT, Llama, Gemma and Qwen are now able to understand and generate natural text, answer questions, summarize documents, translate between different languages and assist users in numerous complex activities.

This versatility stems from the combination of three factors: large-scale neural architectures, enormous amounts of training data, and computational resources of ever-increasing scale. Unlike earlier NLP systems, designed to perform a single specific task, LLMs learn a general representation of language that can subsequently be adapted to many applications, often without the need for dedicated training for each of them.

2.1.1 The Transformer Architecture

Almost all modern Large Language Models are based on the *Transformer* architecture, introduced by Vaswani et al. [1]. This architecture marked a turning point with respect to earlier recurrent neural networks, thanks to the introduction of the *self-attention* mechanism, which allows the model to analyze all positions of a sequence simultaneously, overcoming the sequential processing constraints typical of recurrent models.

Conceptually, three distinct representations are computed for each input token — *query* (Q), *key* (K) and *value* (V) — and the attention output is obtained as a weighted combination of the values, where the weights are determined by the similarity between queries and keys:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{2.1}$$

where d_k is the dimension of the key vectors, used as a normalization factor. This mechanism allows the model to assign a different weight to each token in the context when generating a new word, effectively capturing both local and long-range relationships within the text.

A further advantage of the Transformer over recurrent architectures concerns training: since attention operates on all positions of the sequence simultaneously, training operations are highly parallelizable, making it possible to build models with billions of parameters on modern hardware. For these reasons, the Transformer is today the reference architecture for most large language models.

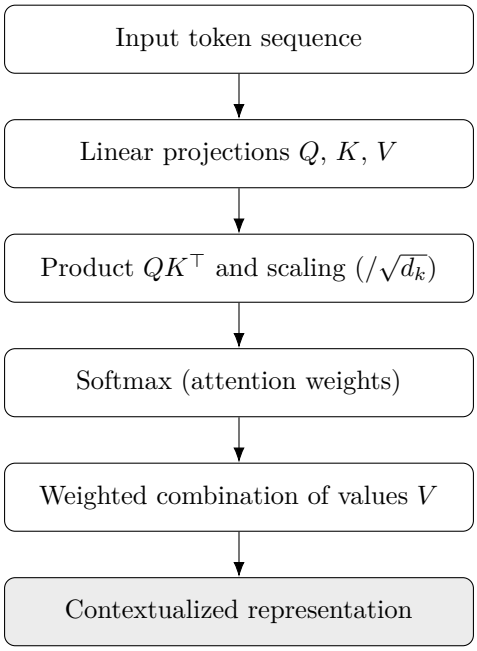


Figure 2.1. Schematic representation of the self-attention mechanism: the projections Q, K and V are derived from the input sequence; the product $QK^\top$, suitably rescaled and normalized via softmax, determines the weights with which the values V are combined to produce the contextualized output.

2.1.2 Pre-training

The initial phase in the training of a Large Language Model is known as *pre-training*. In this phase the model is trained on enormous collections of text drawn from books, web pages, scientific articles and other textual sources, with the goal of learning the statistical regularities of natural language.

The most common training objective is next-token prediction within a sequence, which can be formalized as the maximization of the log-likelihood of the training data:

$$\mathcal{L}_{\text{pre-training}}(\theta) = \sum_{i=1}^{N} \log P_\theta(t_i \mid t_1, \ldots, t_{i-1}) \tag{2.2}$$

where $t_1, \ldots, t_N$ is the token sequence and θ denotes the model parameters. By repeating this process over billions of examples, the model gradually acquires syntactic, semantic and, to some extent, factual knowledge, developing an internal representation of language general enough to be reused in many different contexts.

At the end of pre-training the model therefore possesses broad linguistic capabilities, but is not yet specifically optimized to follow instructions or to interact

effectively with a user: its natural tendency is, in fact, to produce a plausible continuation of a text, not to respond to a request.

2.1.3 Instruction Tuning

To make models suitable for conversational applications, a second training phase, known as *instruction tuning*, is generally carried out. In this phase the model is further trained on datasets consisting of instruction–response pairs, with the goal of learning to correctly follow user requests rather than merely continuing the input text.

Compared to pre-training, instruction tuning employs significantly smaller datasets, but ones made up of carefully selected or annotated data. This procedure turns a general-purpose language model into an *instruction-following* model, capable of producing more pertinent, coherent and useful responses during interaction.

The models employed in this work, Gemma 2 2B Instruct and Qwen 2.5 0.5B Instruct, belong to this category and constitute the starting point of the fine-tuning experiments described in the following chapters.

2.1.4 Limitations of Pre-trained Models

Pre-training and instruction tuning, however effective, do not guarantee that the generated responses are always correct, reliable or consistent with one another. A model may produce inaccurate information (*hallucinations*), contradict itself within the same conversation, or favor formulations that are fluent but factually inaccurate.

These limitations are partly intrinsic to the training objectives described above: neither next-token prediction nor supervised instruction tuning provides the model with an explicit signal about *how much* a response is useful, safe or preferable to an alternative — that is, they distinguish a plausible continuation from a good response, but do not quantify the relative quality among several correct responses. Closing this gap is the goal of preference-based *alignment* techniques, discussed in Section 2.3.

2.2 Fine-Tuning of Large Language Models

As discussed in Section 2.1, pre-training allows Large Language Models to acquire a general knowledge of natural language, while instruction tuning adapts their behavior to the execution of instructions. The models resulting from these two phases are not necessarily optimized for specific tasks or particular application domains: for this reason, a further adaptation phase, known as *fine-tuning*, is often necessary.

Fine-tuning consists in updating the parameters of a pre-trained model using a dataset specific to the task of interest, thus avoiding repeating the costly pre-training process. In the context of Large Language Models, fine-tuning is employed for numerous purposes, including adaptation to specialized domains, improvement of instruction-following ability and, as we shall see in Section 2.3, alignment of the model's behavior with user preferences. The techniques presented in this section are the methodological tools also employed in that context: what changes, in the

following chapters, is the nature of the training signal — no longer a single correct response, but a comparison between alternative responses.

2.2.1 Full Fine-Tuning

The most direct approach to fine-tuning consists in updating all the parameters of the model during training. This strategy, commonly called *full fine-tuning*, allows the model to adapt completely to the new task, but has a very high computational cost.

Modern Large Language Models can contain from hundreds of millions to hundreds of billions of parameters: fully updating such models requires high-performance GPUs, large amounts of video memory and considerable training times, making this solution often impractical in research settings with limited resources. Moreover, for each new task a complete copy of the model weights must be stored, with a consequent increase in the required storage space.

2.2.2 Parameter-Efficient Fine-Tuning

To overcome these limitations, several *Parameter-Efficient Fine-Tuning* (PEFT) techniques have been developed, whose goal is to reduce the number of parameters updated during training while maintaining performance comparable to full fine-tuning.

The idea underlying these approaches is to keep the weights of the pre-trained model frozen and to update only a small number of additional parameters. This reduces both memory consumption and the time required for training, making fine-tuning possible even on relatively modest hardware. Among the numerous families of PEFT techniques proposed in the literature — which include, for example, approaches based on *prompt tuning* or on additional modules (*adapter layers*) — the most widespread one for Large Language Models is currently *Low-Rank Adaptation* (LoRA), described in the next section.

2.2.3 Low-Rank Adaptation (LoRA)

LoRA, introduced by Hu et al. [2], is based on the observation that the weight update needed to adapt a model to a new task typically has a very low intrinsic rank, even when the original weight matrices are of high dimension.

On the basis of this observation, instead of directly updating a weight matrix $W_0 \in \mathbb{R}^{d\times k}$, LoRA keeps it frozen and introduces an update ΔW expressed as the product of two matrices of reduced rank r, called *adapters*:

$$W = W_0 + \Delta W = W_0 + BA, \qquad B \in \mathbb{R}^{d\times r},\ \ A \in \mathbb{R}^{r\times k},\ \ r \ll \min(d,k) \tag{2.3}$$

During training only the parameters of A and B are updated, while W_0 remains frozen. Since the rank r is generally much smaller than the dimensions of the original matrices, the number of trainable parameters is drastically reduced: in practice, updating less than 1% of the model's total parameters is often sufficient to obtain results competitive with full fine-tuning.

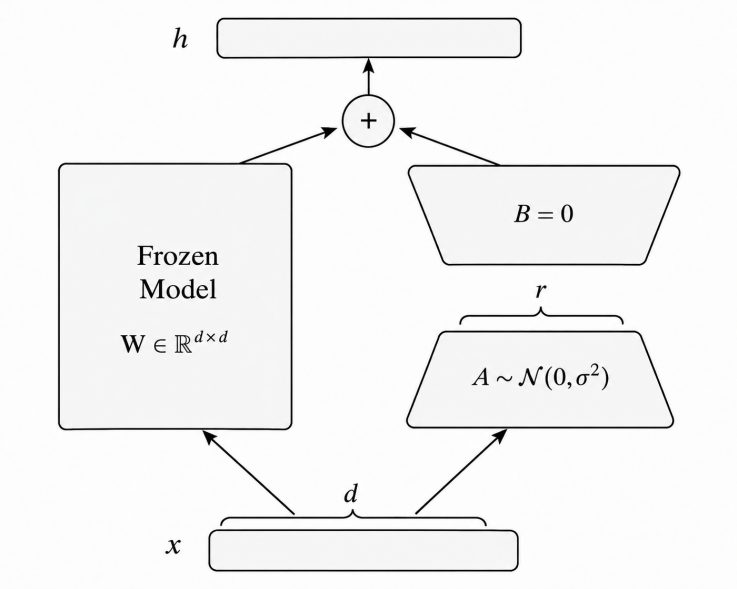


Figure 2.2. Diagram of how LoRA works. The pre-trained weights **W** remain frozen, while the update $\Delta W = BA$ is learned through two matrices of reduced rank r. Matrix A is randomly initialized, while B is initialized to zero. The final output h is obtained by summing the contribution of the pre-trained model and that of the LoRA adaptation.

A further advantage of this approach lies in the possibility of distributing the adapters A and B separately, given their small size, avoiding the need to store or share a complete copy of the model for each learned task.

2.2.4 QLoRA

QLoRA (Quantized Low-Rank Adaptation), proposed by Dettmers et al. [3], extends the LoRA approach by combining it with quantization of the base model, further reducing the memory requirements for fine-tuning.

Specifically, QLoRA keeps the weights of the pre-trained model in a 4-bit quantized format (typically the *NormalFloat4* format, NF4), while the LoRA adapters are kept and updated at higher precision. During the forward pass the quantized weights are temporarily dequantized for computation, but remain stored in compressed format, significantly reducing video memory consumption compared to loading the model in full or mixed precision.

This strategy makes it possible to fine-tune models with billions of parameters even on GPUs with limited memory, such as the T4 GPUs used for the experiments described in the following chapters. In this work, all fine-tuning experiments were carried out using QLoRA, relying on the Unsloth library [4] for loading the quantized models and managing the LoRA adapters; implementation details and the hyperparameters used are described in Chapter 4.

2.3 Preference-Based Alignment

As discussed in Section 2.1.4, pre-training and instruction tuning do not provide the model with an explicit signal about the relative quality of alternative responses. Alignment is the process by which a model's behavior is modified so that the generated responses are more consistent with human preferences, thereby addressing precisely the dimension left uncovered by the earlier training phases.

The goal of alignment is therefore not to increase the amount of knowledge learned by the model, but to steer its behavior toward responses considered more correct, useful and safe. Unlike traditional supervised learning, in which a single correct response is available for each input, alignment problems are generally addressed through *preference datasets*. In this type of dataset, two or more responses to the same prompt are compared with each other; an annotator, human or automatic, indicates which response is preferable according to given quality criteria.

The preferences collected in this way constitute a richer learning signal than plain reference text, since they allow the model to learn not only *what* to answer, but also *which* response is to be considered better among several alternatives. The paradigm that has contributed most to the spread of this approach is *Reinforcement Learning from Human Feedback* (RLHF), described in the next section.

2.3.1 Reinforcement Learning from Human Feedback

The idea underlying RLHF is to use human judgment as a learning signal. After the pre-training and instruction tuning phases described in Section 2.1, the model is used to generate multiple responses to the same prompt. These responses are then compared by human annotators, who are asked to indicate which response is preferable according to criteria such as correctness, completeness, clarity and safety.

The collected preferences are then used to build an optimization pipeline articulated in two successive phases: the training of a reward model, described in Section 2.3.2, and a subsequent reinforcement learning phase based on the Proximal Policy Optimization (PPO) algorithm, aimed at updating the language model and described in Section 2.3.3.

Figure 2.3 schematically summarizes the workflow of the RLHF paradigm.

2.3.2 The Reward Model

To be able to guide the update of the language model according to the collected preferences, RLHF involves the training of an auxiliary model, called *reward model*, whose task is to assign a scalar score to each generated response, thus making the quality judgment expressed by the annotators explicit and quantifiable.

The reward model is typically obtained from the language model itself, by replacing the final vocabulary generation head with a regression head that produces a single scalar value. Its training is based on the pairwise comparisons collected during annotation: given a pair of responses (y_w, y_l) generated from the same prompt x, where y_w denotes the preferred response (*winner*) and y_l the rejected one (*loser*), the model is trained so that y_w receives a higher score than y_l.

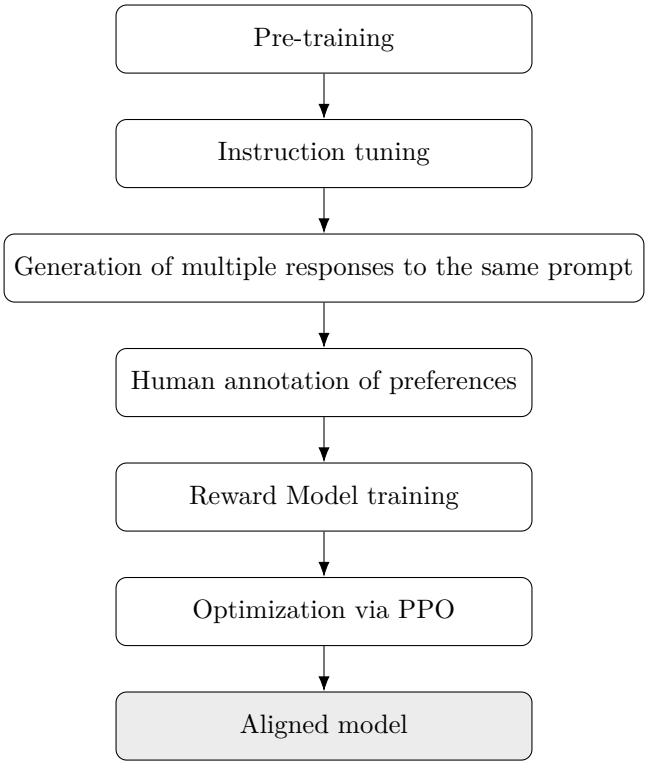


Figure 2.3. Workflow of the Reinforcement Learning from Human Feedback (RLHF) paradigm: starting from the pre-trained, instruction-tuned model, multiple responses are generated and subsequently annotated by human raters; the collected preferences are used to train a reward model, whose signal finally guides the optimization of the language model via PPO.

This objective is formalized through the Bradley-Terry model [5], according to which the probability that y_w is preferred to y_l is a function of the difference between their respective scores:

$$P(y_w \succ y_l \mid x) = \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big) \tag{2.4}$$

where r_ϕ denotes the reward model parameterized by ϕ and σ the sigmoid function. The parameters ϕ are optimized by minimizing the negative log-likelihood of this probability over the entire preference dataset.

Once trained, the reward model is used as a reward function in the subsequent reinforcement learning phase, providing the language model with a learning signal that approximates human judgment without requiring the direct involvement of annotators for each newly generated response.

2.3.3 Proximal Policy Optimization (PPO)

Once the reward model is available, the final phase of RLHF consists in updating the parameters of the language model — called the *policy* in this context — so as to maximize the expected reward according to the trained reward model. This step is generally tackled with reinforcement learning algorithms, the most widespread of which is *Proximal Policy Optimization* (PPO), proposed by Schulman et al. [6].

PPO is an on-policy algorithm that updates the policy incrementally, limiting the size of the updates at each iteration through a *clipped surrogate objective*. This mechanism prevents excessive changes in the policy between one update and the next, helping to stabilize a training process that is typically unstable when applied to large action spaces such as those of Large Language Models.

In the context of RLHF, the objective optimized by PPO is not limited to maximizing the reward provided by the reward model, but generally includes a penalty term based on the Kullback-Leibler (KL) divergence between the updated policy and a reference policy, usually coinciding with the model obtained at the end of instruction tuning:

$$\max_{\pi_\theta} \mathbb{E}_{x,\, y\sim\pi_\theta}\left[r_\phi(x,y)\right] - \beta\,\mathrm{KL}(\pi_\theta(y \mid x) \,\|\, \pi_{\mathrm{ref}}(y \mid x)) \tag{2.5}$$

where π_θ is the policy being trained, π_{ref} the reference policy and β a coefficient that controls the strength of the penalty. This term prevents the model from straying too far from the linguistic behavior learned in the earlier phases in order to maximize the reward — a phenomenon known as *reward hacking*.

Training with PPO therefore requires keeping several models in memory simultaneously: the policy being trained, the reference policy, the reward model and, often, an additional *value model* used to estimate the advantage function. This requirement entails a considerable increase in implementation complexity and in the overall computational cost of the pipeline.

2.3.4 Limitations of RLHF

Despite its demonstrated effectiveness in numerous conversational systems, RLHF has several limitations that complicate its adoption, especially in settings with limited computational resources.

First, the pipeline is computationally demanding: as discussed in Section 2.3.3, the reinforcement learning phase requires keeping several models of comparable size in memory simultaneously, with a consequent significant increase in memory consumption compared to a single model.

Second, the process requires *online* generation of responses by the policy during training, which must then be evaluated by the reward model. This requirement introduces additional computational overhead and makes the whole procedure slower than traditional supervised fine-tuning approaches.

Finally, training via reinforcement learning is known for its instability: small variations in the hyperparameters can lead to very different results, and the process is generally more sensitive to problems such as reward hacking or policy collapse than more direct optimization techniques.

These limitations have motivated the search for alternative approaches capable of achieving alignment results comparable to RLHF while reducing implementation complexity and computational requirements. Among these, one of the most relevant proposals is *Direct Preference Optimization* (DPO), described in the next section.

2.4 Direct Preference Optimization

As discussed in Section 2.3.4, the limitations of RLHF — computational cost, the need for online generation and the instability of training via reinforcement learning — have motivated the search for alternative approaches capable of achieving comparable alignment results while reducing implementation complexity. *Direct Preference Optimization* (DPO), proposed by Rafailov et al. [7], is one of the most relevant solutions in this direction.

2.4.1 The Idea Behind DPO

The central observation behind DPO concerns the constrained optimization objective employed in PPO (Section 2.3.3):

$$\max_{\pi_\theta} \; \mathbb{E}_{x,\, y\sim\pi_\theta}\big[r_\phi(x,y)\big] - \beta\, \mathrm{KL}\big(\pi_\theta(y \mid x) \,\|\, \pi_{\mathrm{ref}}(y \mid x)\big)$$

For a fixed reward function r, this objective admits a closed-form solution. The optimal policy π^* that maximizes it can indeed be expressed as:

$$\pi^*(y \mid x) = \frac{1}{Z(x)}\, \pi_{\mathrm{ref}}(y \mid x)\, \exp\!\left(\frac{1}{\beta} r(x,y)\right) \tag{2.6}$$

where $Z(x) = \sum_y \pi_{\mathrm{ref}}(y \mid x) \exp\!\left(\frac{1}{\beta} r(x,y)\right)$ is a normalization constant that depends exclusively on the prompt x, and not on the response y.

By inverting this relation, the reward can be expressed as a function of the optimal policy:

$$r(x,y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\mathrm{ref}}(y \mid x)} + \beta \log Z(x) \tag{2.7}$$

This step is crucial: the reward can be rewritten entirely in terms of the policy itself, without the need for a separate reward model. Substituting this expression into the Bradley-Terry model introduced in Section 2.3.2, the term $Z(x)$ cancels out in the difference between the rewards of y_w and y_l (since it depends only on x, which is identical for both responses), yielding:

$$P(y_w \succ y_l \mid x) = \sigma\!\left(\beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\mathrm{ref}}(y_w \mid x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\mathrm{ref}}(y_l \mid x)}\right) \tag{2.8}$$

This expression directly links the preference probability to the policy, without any further explicit reference to a reward function. DPO exploits exactly this property: instead of training a reward model and subsequently optimizing the policy via reinforcement learning, the policy π_θ can be trained directly on the preference data, by minimizing the negative log-likelihood of the previous relation:

$$\mathcal{L}_{\mathrm{DPO}}(\theta) = -\,\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log \sigma\!\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\mathrm{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\mathrm{ref}}(y_l \mid x)}\right)\right] \tag{2.9}$$

Training thus reduces to a binary classification problem over pairs of responses, solvable with a single supervised optimization pass, similarly to standard fine-tuning.

2.4.2 Dataset Structure: Prompt, Chosen and Rejected

Consistently with the previous formulation, a preference dataset for DPO consists of triplets (x, y_w, y_l), where x is the prompt, y_w the preferred response and y_l the rejected one. In the literature and in the most widespread implementations — including the libraries used in this work — these elements are commonly referred to as *prompt*, *chosen* and *rejected*, a terminology that will be adopted in the following chapters.

It is important to note that this structure coincides exactly with the one required for training the reward model in RLHF (Section 2.3.2): in both cases, one starts from pairwise comparisons between responses, annotated by a human or automatic judge. The difference between the two approaches therefore lies not in the nature of the required data, but in how these data are used: whereas RLHF employs them to train an intermediate reward model, DPO uses them to directly update the parameters of the policy.

2.4.3 Advantages over RLHF

The reformulation proposed by DPO brings several practical advantages over the RLHF pipeline described in Section 2.3.

First, DPO eliminates the need to train a separate reward model: the policy π_θ is updated directly on the preference data, reducing the number of models to be kept in memory compared to the set described in Section 2.3.3. DPO in fact requires only the policy being trained and the reference policy — the latter, moreover, usable in inference-only mode, without gradient computation.

Second, DPO does not require any *online* generation of responses during training: the entire procedure operates offline on a preference dataset fixed in advance, analogously to standard supervised training. This eliminates the overhead associated with iterative sampling from the policy, which is characteristic of PPO.

Finally, since the DPO objective reduces to a differentiable loss function in closed form, training is more stable and simpler to implement than a reinforcement learning loop, avoiding issues such as the reward hacking or policy collapse discussed in Section 2.3.4. Rafailov et al. [7] also report performance comparable or superior to RLHF on several alignment tasks, at a much lower implementation complexity.

2.4.4 Why DPO Is Suitable for This Work

The characteristics of DPO discussed in this section make it particularly suitable for the experimental setting of this work, which is characterized by limited computational resources and the use of small models.

First, the absence of a separate reward model and of an online generation loop significantly reduces video memory requirements, a particularly stringent constraint in the Kaggle environment with T4 GPUs used for the experiments described in Chapter 4. Second, DPO combines naturally with the efficient fine-tuning techniques described in Section 2.2: since the policy being trained is the only model whose parameters need updating, QLoRA can be applied directly to its training, preserving the memory advantages discussed above.

Finally, and more directly, DPO requires as its only input a preference dataset of the form (prompt, chosen, rejected). The relative judgments produced by AutoBench through reciprocal evaluation among models, although not natively structured in this form, lend themselves naturally to being transformed into preference triplets through the extraction pipeline described in Section 2.5, without requiring the training of an intermediate reward model or any other additional processing. This compatibility between the nature of the available data and the requirements of the alignment algorithm motivated the choice of DPO as the fine-tuning method for the experiments described in Chapters 4 and 5.

2.5 Peer-Generated Training Data

As discussed in Section 2.4, training via DPO requires a preference dataset of the form (prompt, chosen, rejected). In the original formulation of RLHF, these annotations are collected from human raters: a process that guarantees high-quality judgments, but is costly, slow and hard to scale to large datasets. For this reason, in recent years there has been growing interest in automatic methods for building preference datasets, in which human judgment is partially or totally replaced by that of other language models.

2.5.1 LLM-as-a-Judge

The *LLM-as-a-Judge* paradigm consists in using a Large Language Model to evaluate the responses generated by other models, in place of human judgment. Zheng et al. [8] showed that large models, suitably instructed through prompting, are able to assess the quality of textual responses with a level of agreement with human judgments comparable to that observed among different human annotators.

This approach typically relies on a single "judge" model, usually larger and more capable than the generator models, which is asked to compare two or more responses according to explicit criteria (correctness, completeness, helpfulness) and to indicate which one is preferable. The main advantage of this approach lies in its scalability: once the evaluation prompt has been defined, an arbitrarily large number of comparisons can be annotated automatically, without the costs and time associated with human annotation.

2.5.2 Peer Assessment

A variant of this paradigm, called *peer assessment* or *peer review*, entrusts the evaluation of responses not to a single, more capable judge model, but to one or more models of a scale comparable to those that generated the responses themselves — sometimes the very same generator models, called upon to evaluate each other's outputs. Li et al. [9] proposed a framework of this kind, showing how the combination of judgments from several "peer" models (*peer rank*), possibly complemented by a discussion phase among the models themselves, can mitigate some of the biases typical of evaluation entrusted to a single judge.

This approach differs from LLM-as-a-Judge in the strict sense in the absence of a judge model of higher quality than the generators: evaluation instead emerges

from the mutual comparison among models of comparable competence, in analogy with the peer review processes employed in academia.

2.5.3 Automatic Preference Datasets

The use of automatic judges, according to either of the two paradigms described, makes it possible to build preference datasets at scale without resorting to human annotators. A relevant example is UltraFeedback [10], a preference dataset built by collecting responses generated by numerous models for a broad set of prompts and evaluating them automatically through a judge model, according to multiple criteria such as helpfulness, honesty and harmlessness.

These datasets share a common structure: a set of prompts, a set of generator models that produce candidate responses, and a procedure — automatic or semi-automatic — that compares these responses and derives labeled (chosen, rejected) pairs from them, directly usable for training via DPO.

2.5.4 Advantages and Limitations

The use of automatic judges to build preference datasets offers significant advantages in terms of scalability and cost. It is in fact possible to generate an arbitrarily large number of comparisons without the time and costs associated with recruiting and training human annotators, making it feasible to build preference datasets even in research settings with limited resources, such as that of this work.

This approach does, however, have some limitations. First, automatic judgments are subject to biases documented in the literature, such as the tendency to prefer longer responses regardless of their actual quality (*verbosity bias*), a sensitivity to the order in which the responses are presented (*position bias*), or, when the judge model belongs to the same family as one of the generators, a tendency to prefer responses stylistically similar to its own (*self-preference bias*). Second, the overall quality of the resulting dataset depends entirely on the quality of the judge employed: any errors or systematic biases in the evaluation propagate directly into the annotated preferences, and hence into the behavior learned by the model during fine-tuning. Finally, in the specific case of *peer assessment*, the absence of a judge of higher quality than the generators may entail the risk that limitations shared among the models involved (for example common stylistic biases or knowledge gaps) go undetected, as they are not recognized as such by any of the evaluators.

2.5.5 Introduction to AutoBench

This work falls within this line of research, adopting the *peer assessment* paradigm for the construction of the preference dataset used in the fine-tuning experiments. To this end, *AutoBench* [11] was used, an automatic evaluation framework in which a set of language models generates, evaluates and mutually compares their respective responses, producing an aggregate measure of their quality.

In addition to providing a ranking of the participating models, AutoBench makes available the evaluations assigned to individual responses. This information forms the basis for the automatic construction of preference datasets, transforming the scores obtained during the *peer assessment* process into `(chosen, rejected)` pairs

compatible with preference-based alignment algorithms such as Direct Preference Optimization.

In this work, the data produced by AutoBench were subjected to a preprocessing phase aimed at building the training datasets used in the experiments. Details of the framework and of the entire data preparation pipeline are described in Chapter 3.

Chapter 3

AutoBench and the Construction of Preference Datasets

As discussed in Chapter 2, this work uses AutoBench [11] as the source of the preference data employed for the fine-tuning of language models. Although the framework was originally designed as an automatic evaluation benchmark based on the *peer assessment* paradigm, the information produced during the evaluation process can be reused to build preference datasets compatible with alignment algorithms such as Direct Preference Optimization (DPO).

This chapter describes the structure of the data provided by AutoBench and the pipeline developed to transform these data into training datasets. It also illustrates the cleaning, filtering, deduplication and preference pair extraction procedures adopted in this work.

3.1 AutoBench as a Source of Peer-Generated Data

Training language models with preference-based alignment techniques, such as Direct Preference Optimization (DPO), requires datasets containing pairs of responses ordered according to a preference relation. Traditionally, such datasets are built through human annotation processes, in which one or more raters select, for each prompt, the response deemed best among several alternatives. Although this approach guarantees high quality, it is costly in terms of time and financial resources, limiting the possibility of generating large datasets.

In this work, we chose to explore an alternative approach based on the use of preference data generated automatically by other language models. In particular, we used AutoBench, an automatic evaluation framework in which the participating LLMs simultaneously take on the roles of task generators, contestants and evaluators. Through a process of mutual evaluation, each model assigns a score to the responses produced by the other participants, contributing to the construction of a collective performance ranking.

The aspect of greatest interest for this thesis is not the final ranking produced by AutoBench, but rather the data generated during the evaluation process. For each prompt, the responses provided by the different models are in fact available, together with the scores obtained through the *peer assessment* mechanism. This

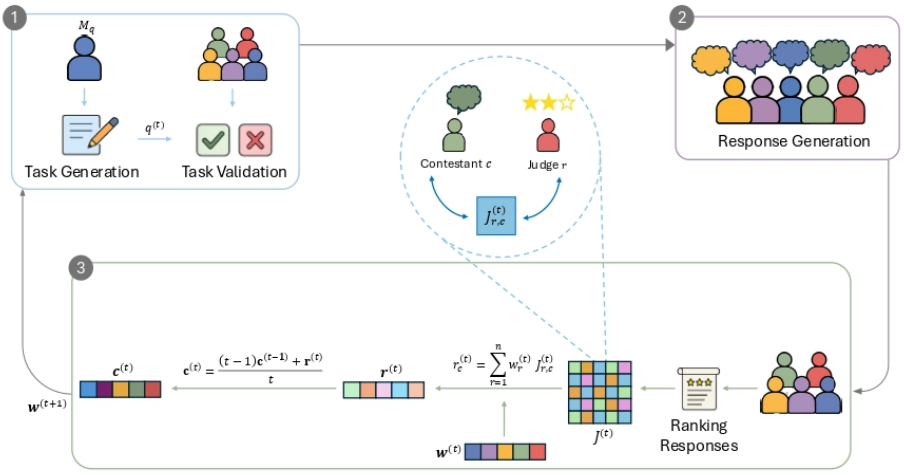


Figure 3.1. Diagram of the evaluation process adopted by AutoBench: task generation and validation, response generation by the contestant models and mutual evaluation by the judge models, with iterative updating of the judgment weights. Figure reproduced from [11].

information constitutes a natural source of preference data, as it makes it possible to identify responses judged better or worse than others without resorting to additional human annotations.

Starting from these observations, the original AutoBench dataset was reinterpreted as a collection of implicit preferences among responses generated by different models. The goal of the process described in this chapter is therefore to transform the raw data produced by AutoBench into datasets compatible with training via Direct Preference Optimization, preserving as much as possible the preference signal deriving from the collective evaluation of the participating models.

3.2 Data Preparation and Cleaning

For each evaluated prompt, the dataset obtained from AutoBench contains the responses generated by the participating models together with the corresponding scores resulting from the *peer assessment* process. Before proceeding to the extraction of preference pairs, it was necessary to carry out a preliminary data preparation and cleaning phase, aimed at ensuring the quality and consistency of the information used for fine-tuning.

First, only the *open-source* models present in the benchmark were retained. This choice was motivated by the desire to build a dataset derived entirely from freely accessible and reproducible models, avoiding the influence of proprietary systems not available to the research community. Following the removal of the responses generated by *closed-source* models, the corresponding scores expressed during the evaluation phase were also removed from the judgment dictionaries. Since the original aggregation algorithm used by AutoBench is not available, the final score of each response was recomputed as the arithmetic mean of only the judgments coming from the remaining *open-source* models.

Table 3.1. Open-source models selected from the AutoBench run used in this thesis.

Family	Model
DeepSeek	DeepSeek-R1-0528
DeepSeek	DeepSeek-v3.2
DeepSeek	DeepSeek-v3.2-speciale
GLM	GLM-4.5-Air
GLM	GLM-4.6
Kimi	Kimi-k2-0905
Kimi	Kimi-k2-thinking
Nemotron	Llama-3.3-Nemotron-Super-49B-v1.5
MiniMax	MiniMax-M2
Mistral	Mistral-Large-2512
Mistral	Mistral-Medium-3.1
Mistral	Ministral-8B-2512
Nemotron	Nemotron-3-Nano-30B-A3B
Nemotron	Nemotron-Nano-9B-v2
Olmo	Olmo-3.1-32B-Think
Qwen	Qwen3-235B-A22B-Thinking-2507
Qwen	Qwen3-235B-A22B-2507
Qwen	Qwen3-Next-80B-A3B-Thinking
GPT-OSS	GPT-OSS-20B
GPT-OSS	GPT-OSS-120B

In the AutoBench framework, each evaluator model assigns the competing responses a score on a discrete scale from 1 to 5, where higher values indicate higher perceived quality. The resulting mean score, indicated in the dataset by the column `average_rank_os`, is the basis for the subsequent preference pair extraction phases described in Section 3.3.

Table 3.1 lists all the models retained in the dataset construction phase.

A text normalization phase was then carried out using the `ftfy` library (*Fixes Text For You*), used to automatically correct any character encoding problems present in the dataset. In particular, cases of *mojibake* were identified and corrected, that is, corrupted character sequences generated by conversion errors between different text encodings. This operation made it possible to standardize the content of the columns containing prompts and responses, reducing the risk of introducing noise during the subsequent processing phases. Figure 3.2 shows some representative examples of the corrections made.

All instances with missing information in the essential columns (`question`, `answer` and `average_rank_os`) were also removed. To further limit the presence of anomalous examples, a filter on response length was applied. In particular, only responses containing between 3 and 2000 words were retained. This choice made it possible to exclude both empty or extremely short responses and exceptionally long generations potentially attributable to errors or anomalous model behavior.

At the end of this phase, a clean and consistent dataset was obtained, which served as the basis for the subsequent extraction of the preference pairs needed for

Examples of *mojibake* correction with `ftfy`

Before

```
- Drive allâ€reduce *P99* latency down...
```

After

```
- Drive all-reduce *P99* latency down...
```

Before

```
...Metaâ€s 10k-GPU...
```

After

```
...Meta's 10k-GPU...
```

Before

```
...medium-sized â€œelephantâ€ gradient flows...
```

After

```
...medium-sized "elephant" gradient flows...
```

Figure 3.2. Examples of automatic correction of encoding errors present in the dataset.

training via Direct Preference Optimization.

Moreover, the goal of this work is to assess the possibility of generating training data exclusively from open resources, making the entire pipeline replicable without depending on proprietary models.

Figure 3.3 schematically summarizes the data preparation and cleaning phases just described, from the raw dataset produced by AutoBench to the clean dataset used as the basis for the extraction of preference pairs.

3.3 Extraction of Preference Pairs

The final goal of the data preparation process is the construction of datasets compatible with training via *Direct Preference Optimization* (DPO). This methodology requires, for each training example, a triplet consisting of a prompt, a preferred response (*chosen*) and a non-preferred response (*rejected*). Since the original AutoBench dataset contains multiple responses associated with the same prompt and the corresponding scores resulting from the peer evaluation process, it was necessary to define a procedure to convert this information into explicit preference pairs.

In this work, two different preference extraction strategies were explored. Both rely on the assumption that the mean score assigned by AutoBench represents a reliable estimate of the relative quality of the responses and can therefore be used to establish a preference relation between two different generations.

3.3.1 Best-vs-Worst Strategy

The first strategy, called "Best-vs-Worst", involves selecting a single preference pair for each prompt. For each set of responses associated with the same question, the response with the highest mean score and the one with the lowest mean score are

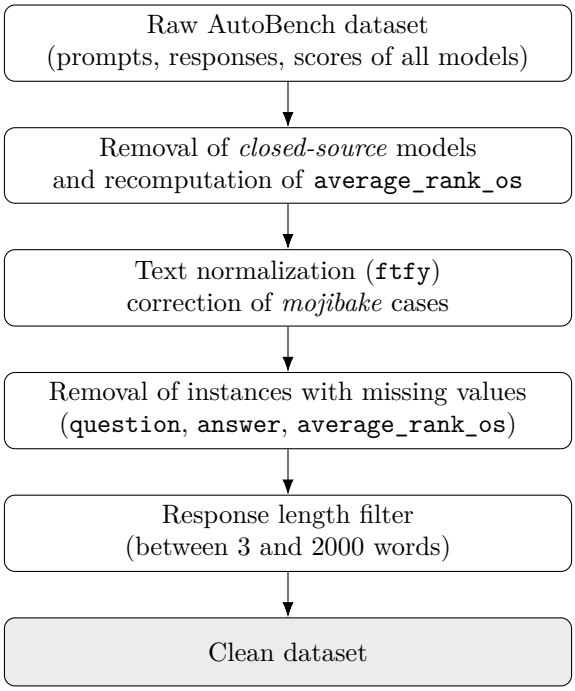


Figure 3.3. Data preparation and cleaning pipeline: starting from the raw dataset produced by AutoBench, the *closed-source* models are removed, text encoding errors are corrected, instances with missing values in the essential columns are eliminated and responses are filtered by length, yielding the clean dataset used in the subsequent phases.

identified. The former is assigned to the *chosen* field, while the latter is assigned to the *rejected* field.

Formally, given a set of responses $R = \{r_1, r_2, ..., r_n\}$ associated with a prompt q, with corresponding scores s_i, the following are selected:

$$r_{chosen} = \arg\max_{r_i \in R} s_i$$

$$r_{rejected} = \arg\min_{r_i \in R} s_i$$

This strategy produces a relatively compact dataset, but one characterized by a particularly strong preference signal, since it maximizes the distance between the response considered best and the one considered worst for each prompt.

3.3.2 Pairwise Strategy

The second strategy, called "Pairwise", instead aims to exploit a larger portion of the information contained in the ranking produced by AutoBench. For each prompt, all possible pairs of responses generated by the participating models are considered.

Given a pair of responses (r_i, r_j), their respective mean scores are compared. The response with the higher score is assigned to the *chosen* field, while the one with the lower score is assigned to the *rejected* field.

The difference between the scores of the two responses is defined as:

$$\Delta_{ij} = |s_i - s_j|$$

The resulting pairs are then subjected to the quality filters described in Section 3.4.

The Pairwise approach yields a significantly larger number of examples than the Best-vs-Worst strategy, increasing the amount of data available for fine-tuning. However, it requires additional quality control mechanisms to avoid redundant or poorly informative pairs, described in the next section.

3.4 Quality Filters

Directly applying the strategies described in the previous section produces a large number of preference pairs, not all equally informative for training. In particular, some pairs may be characterized by negligible quality differences, others may contain nearly identical responses, while still others may be overrepresented because of the large number of combinations generated from the same prompt.

To mitigate these issues, several filtering mechanisms were introduced, with the aim of increasing the overall quality of the dataset and preserving a preference signal that is as meaningful as possible.

3.4.1 Preference Gap Filter

For each candidate pair, the absolute difference between the mean scores assigned by AutoBench to the two responses is computed:

$$\Delta = |s_{chosen} - s_{rejected}|$$

Pairs with a value of Δ below a predefined threshold are discarded.

Since the scores are expressed on a scale from 1 to 5, a value of $\Delta = 0.3$ represents a relatively small difference, while $\Delta = 0.5$ corresponds to a more marked gap between the mean ratings obtained by the two responses.

The introduction of this filter is motivated by the observation that very small differences between scores may reflect situations in which the two responses are essentially equivalent according to the collective judgment expressed by the evaluator models.

The threshold Δ is therefore a parameter controlling dataset quality: higher values favor pairs with clearer preferences, while lower values increase the number of available examples at the expense of the strength of the learning signal.

3.4.2 Textual Similarity Filter

Even when the score differences are significant, two responses may be extremely similar in content. In such cases the pair provides limited information to the model, since the distinction between the preferred and the non-preferred response is hardly evident.

To identify these situations, the Jaccard similarity computed on the set of words contained in the two responses was used:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

where A and B represent the sets of words present in the two generations being compared.

Pairs with a similarity above a predefined threshold were removed from the dataset. This procedure reduces the presence of redundant examples and favors learning from truly distinctive comparisons.

Illustrative example of a pair excluded due to high textual similarity

Prompt: Explain the main difference between a `list` and a `tuple` in Python.

Response A (chosen): In Python, a list is a mutable data structure, meaning that its elements can be modified, added or removed after creation. A tuple, on the other hand, is immutable: once created, its content cannot be altered.

Response B (rejected): In Python, a tuple is an immutable data structure, meaning that its content cannot be altered after creation. A list, on the other hand, is mutable: its elements can be modified, added or removed.

Computed Jaccard similarity: $J \approx 0.91$ *(exclusion threshold: 0.85)*

Figure 3.4. Illustrative example of a pair of semantically equivalent responses that differ only in the order in which the concepts are presented. Despite a possible gap in the scores assigned by AutoBench, the high lexical similarity leads to the exclusion of the pair from the final dataset.

3.4.3 Limiting the Number of Pairs per Prompt

In the case of the Pairwise strategy, the number of pairs generated from a single prompt grows rapidly as the number of available responses increases. Consequently, some prompts could contribute a significantly higher number of examples than others, introducing an imbalance in the distribution of the training data.

To prevent specific prompts from dominating the final dataset, a maximum limit on the number of pairs generated for each question was introduced. When the number of valid pairs exceeds this limit, only those with the largest values of Δ are kept.

This choice preserves the most informative preferences, while reducing redundancy and improving the overall balance of the dataset.

3.5 Construction of the Final Datasets

Applying the preference extraction strategies and the quality filters described in the previous sections led to the generation of several datasets intended for the fine-tuning

experiments. The goal was not solely to maximize the number of available examples, but also to study how different construction criteria affect the quality of the learning signal and the performance of the trained models.

The resulting datasets differ both in the preference extraction strategy adopted and in the selectivity of the filters applied. In particular, datasets were built that embody different trade-offs between the amount of available data and the strength of the preference signal.

3.5.1 Best-vs-Worst Dataset

The first dataset was generated using the Best-vs-Worst strategy described in Section 3.3. For each prompt, only the pair consisting of the response with the highest score and the one with the lowest score was retained.

This procedure produces a single training example for each available prompt, favoring signal quality over data quantity. The resulting pairs in fact represent the cases in which the difference in evaluation between the responses is greatest.

The final dataset contains:

- Number of unique prompts: **313**
- Number of DPO triplets generated: **313**
- Mean gap between *chosen* and *rejected*: **1.746**
- Median gap between *chosen* and *rejected*: **1.500**

3.5.2 Pairwise Dataset

Unlike the Best-vs-Worst strategy, the Pairwise approach makes it possible to extract multiple preference pairs for each prompt. Starting from the dataset obtained at the end of the preprocessing phase, all possible pairs of responses were considered and subsequently filtered by applying the criteria described in Section 3.4.

The starting dataset, before applying the filters specific to the Pairwise strategy, contains:

- Number of unique prompts: **313**
- Number of DPO triplets generated: **6187**
- Mean gap between *chosen* and *rejected*: **1.256**
- Median gap between *chosen* and *rejected*: **1.050**

Pairwise Configuration ($\Delta = 0.3$)

For the first configuration, a minimum score gap threshold of $\Delta = 0.3$, a Jaccard similarity threshold of **0.85** and a maximum of **20** pairs selected per prompt were used.

Applying these criteria produces a final dataset of **6097** DPO triplets, with a mean gap of **1.272** and a median gap of **1.067**.

Pairwise Configuration ($\Delta = 0.5$)

The second configuration adopts more restrictive selection criteria, imposing a minimum threshold of $\Delta = 0.5$, keeping the Jaccard similarity threshold unchanged (**0.85**) and reducing the maximum number of pairs selectable per prompt from **20** to **10**.

The joint application of these constraints reduces the number of available examples, yielding a final dataset of **3058** DPO triplets. This reduction is due both to the higher threshold on the score gap, which eliminates pairs with less marked preferences, and to the stricter limit on the maximum number of pairs retained for each prompt. At the same time, the preferences are on average clearer, as shown by the increase in the mean gap (**1.499**) and the median gap (**1.300**).

3.5.3 Comparison Between the Datasets

Table 3.2 summarizes the main characteristics of the generated datasets.

Table 3.2. Characteristics of the final datasets used in the experiments.

Dataset	DPO triplets	Mean gap	Median gap
Best-vs-Worst	313	1.746	1.500
Pairwise ($\Delta = 0.3$)	6097	1.272	1.067
Pairwise ($\Delta = 0.5$)	3058	1.499	1.300

The availability of datasets with different sizes and different levels of selectivity makes it possible to experimentally analyze the impact of data construction choices on the fine-tuning process. In particular, the experiments described in the following chapters will allow us to assess the trade-off between the number of available examples and the quality of the preference signal provided to the model.

3.6 Data Splitting

After the generation of the preference triplets, the datasets were split into a training set and a test set. This split was introduced with a twofold objective: to enable training the models via Direct Preference Optimization and, at the same time, to have a set of examples not observed during fine-tuning to be used for evaluating the model's behavior.

Unlike a random split performed directly on individual triplets, the split was carried out at the prompt level. In other words, all preference pairs generated from the same prompt are assigned entirely to the training set or entirely to the test set.

This choice was adopted to avoid *data leakage*. With a random split of individual triplets, examples derived from the same prompt could in fact appear in both the training set and the test set. In such a situation the model could indirectly benefit from information already observed during training, leading to an overestimation of its actual performance.

For each dataset, 90% of the unique prompts were assigned to the training set, while the remaining 10% were assigned to the test set. The selection was made

randomly but deterministically using a pseudo-random seed fixed at 42, ensuring full reproducibility of the experiments.

The resulting test set was used to evaluate the models' ability to reproduce the preferences present in the data generated by AutoBench. In particular, by comparing the performance obtained before and after fine-tuning, it was possible to measure the actual learning of the preference signal contained in the constructed datasets.

Evaluation on this set of data represents an internal measure of the quality of the alignment obtained. It was subsequently complemented by standardized external benchmarks, described in the following chapters, in order to verify whether learning the preferences also translated into an improvement in the model's general capabilities.

The entire preference dataset construction pipeline adopted in this work is summarized in Figure 3.5. Starting from the raw dataset produced by AutoBench, cleaning and normalization operations are performed, followed by the extraction of preference pairs, the application of the quality filters and the final split into training and test sets.

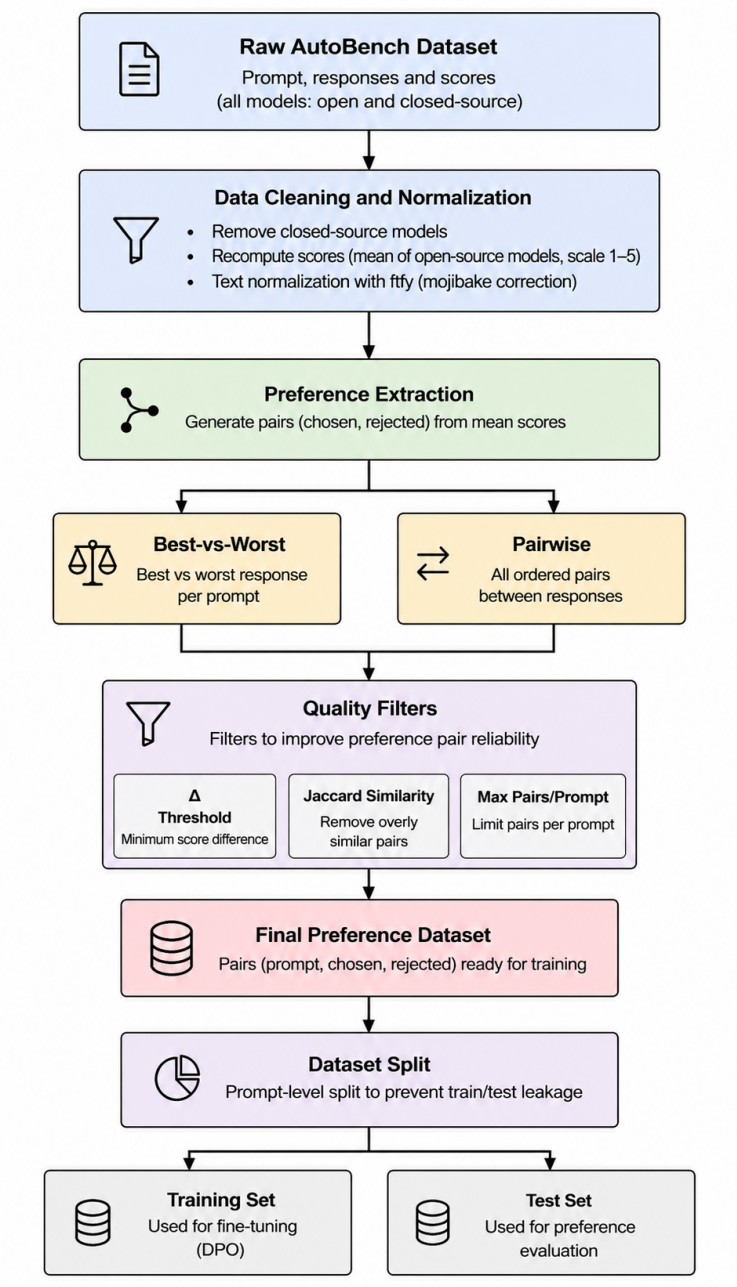


Figure 3.5. Preference dataset construction pipeline used in this work.

Chapter 4

Experimental Setup

4.1 Experimental Objectives

The main objective of this work is to analyze the feasibility of using training data generated through evaluation processes among language models (*peer-generated training data*) for the alignment of Large Language Models via preference-based optimization techniques.

Starting from the data produced by the AutoBench framework, described in Chapter 3, several preference datasets were built in the `prompt, chosen, rejected` format and subsequently used for the fine-tuning of open-source language models with the Direct Preference Optimization (DPO) algorithm.

The experiments were designed to answer the following research questions:

1. Is it possible to use preferences generated exclusively by other language models to fine-tune a target model without resorting to human annotations?
2. To what extent does fine-tuning on datasets derived from AutoBench change the model's performance with respect to the starting version?
3. What impact do different strategies for building preference pairs have on the quality of the final model?
4. Are the improvements observed on the AutoBench-derived dataset also reflected on external benchmarks commonly used for the evaluation of Large Language Models?

The following sections describe the experimental setup and the methodology adopted to answer these questions, providing all the elements needed to interpret the results presented in Chapter 5.

4.2 Experimental Environment

All the experiments presented in this work were run on the Kaggle Notebooks platform, which provides free computational resources based on NVIDIA Tesla T4 GPUs. This choice was motivated by the immediate availability of the infrastructure

and by the possibility of running fine-tuning experiments on small and medium-sized language models without resorting to dedicated hardware.

The experiments were conducted using a single NVIDIA Tesla T4 GPU with 14.6 GB of video memory (VRAM). Because of the available hardware limitations, it was necessary to adopt memory optimization techniques to make it possible to train the models considered.

To this end, the *Unsloth* library was used, a collection of optimizations for the efficient fine-tuning of Large Language Models that integrates modified versions of the main libraries of the Hugging Face ecosystem. In particular, Unsloth provides native support for loading models in 4-bit quantized format, optimized implementations of LoRA and QLoRA, as well as gradient checkpointing mechanisms designed to reduce memory consumption during training.

Since the Tesla T4 GPU does not support the BF16 (*Brain Floating Point*) numerical format, all experiments were run in FP16 (*Half Precision*), which was automatically verified at the start of each experimental notebook.

Table 4.1 summarizes the hardware and software configuration used.

Table 4.1. Configuration of the experimental environment.

Component	**Configuration**
Platform	Kaggle Notebooks
GPU	NVIDIA Tesla T4
GPU memory	14.6 GB VRAM
Numerical precision	FP16
Quantization	4 bit (QLoRA)
Main framework	Unsloth
Training libraries	TRL, Transformers, PEFT
Dataset format	Parquet
Experimental seed	42

The combined use of quantization and *Parameter-Efficient Fine-Tuning* techniques made it possible to train the models considered in this work within the memory limits of the available GPU, allowing the entire experimental campaign to be carried out without dedicated computing infrastructure.

4.3 Fine-Tuning Procedure

All the experiments in this work were conducted using the Direct Preference Optimization (DPO) algorithm, introduced in Chapter 2.

This section describes exclusively the implementation aspects of the fine-tuning procedure adopted, including the QLoRA configuration, the training hyperparameters and the models used.

4.3.1 Quantization and QLoRA

As described in Section 2.2.4, all experiments were run using the QLoRA approach through the Unsloth library on a single NVIDIA Tesla T4 GPU (Section 4.2). The

adapters were applied to the main modules of the Transformer architecture, covering both the attention mechanisms and the feed-forward blocks.

This configuration preserves good model adaptability while significantly reducing the number of trainable parameters and, consequently, memory consumption during training.

The main LoRA hyperparameters were kept constant across all experiments:

- rank (r): 16;
- scaling factor (α): 16;
- dropout: 0;
- bias: `none`.

This configuration represents a good compromise between model adaptability and computational efficiency.

A further parameter relevant to the tokenization procedure is the maximum sequence length (`MAX_SEQ_LENGTH`), which determines the maximum number of tokens considered for each training example, together with the corresponding maximum length reserved for the prompt (`MAX_PROMPT_LENGTH`), set in all experiments to half of `MAX_SEQ_LENGTH`. Since the responses generated during the AutoBench *peer assessment* process are often quite long, this parameter directly affects the portion of text actually seen by the model during training. Because of the memory constraints imposed by the Tesla T4 GPU (Section 4.2), this value was configured differently for the two models considered:

- Gemma 2 2B Instruct: `MAX_SEQ_LENGTH` = 850 tokens, `MAX_PROMPT_LENGTH` = 425 tokens;
- Qwen 2.5 0.5B Instruct: `MAX_SEQ_LENGTH` = 2000 tokens, `MAX_PROMPT_LENGTH` = 1000 tokens.

The lower value adopted for Gemma 2 2B is due to the larger memory footprint of this model compared to Qwen 2.5 0.5B, which made more aggressive truncation of the sequences necessary to stay within the available VRAM limits. The implications of this choice are discussed in Section 6.5.

4.3.2 Training Configuration

Training was carried out using the `DPOTrainer` implementation provided by the TRL library [12], suitably optimized by Unsloth through custom kernels and advanced gradient checkpointing.

The main training hyperparameters were kept constant across all experiments, as shown in Table 4.2.

In addition, the gradient checkpointing provided by Unsloth was used, which allows a significant reduction in memory consumption during training.

As for the number of training epochs, the experiments on the Best-vs-Worst and Pairwise ($\Delta = 0.3$) datasets were run for a single epoch. The exception is the experiment on the Pairwise ($\Delta = 0.5$) dataset, for which the model's behavior was instead analyzed across several successive checkpoints, as described in Chapter 5.

Table 4.2. Hyperparameters used during DPO fine-tuning.

Parameter	Value
Learning rate	5×10^{-5}
Scheduler	Cosine
Optimizer	AdamW 8-bit
Batch size per device	2
Gradient accumulation steps	8
Effective batch size	16
DPO β	0.1
Seed	42

4.3.3 Models Considered

The experiments were conducted on two open-source language models of different sizes:

- Gemma 2 2B Instruct;
- Qwen 2.5 0.5B Instruct.

The choice of relatively compact models was motivated mainly by the available computational constraints. Both models are in fact compatible with running on a single Tesla T4 GPU using QLoRA and 4-bit quantization.

Gemma 2 2B is the largest model used in the study, while Qwen 2.5 0.5B is a particularly lightweight solution, which made it possible to run a larger number of experiments on different datasets.

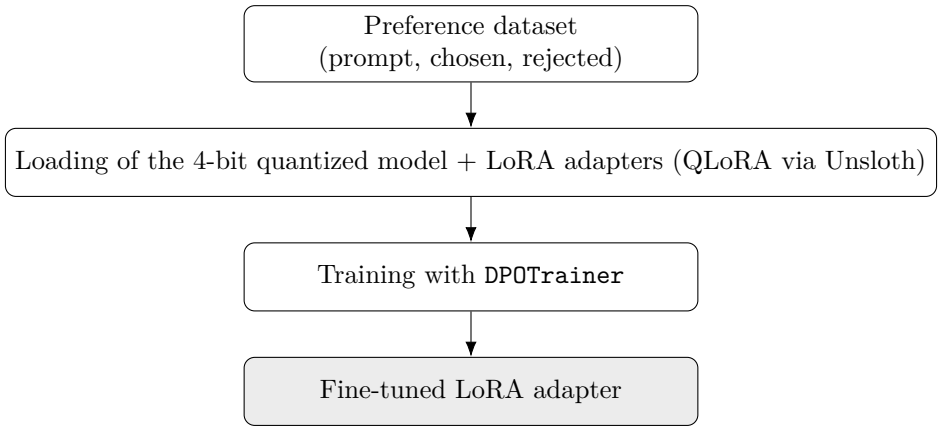


Figure 4.1. Fine-tuning workflow: the preference dataset is used by `DPOTrainer` to train, via QLoRA, the LoRA adapters applied to the 4-bit quantized base model, producing the fine-tuned adapter as output.

4.4 Experiment Configuration

To analyze the effectiveness of the training data generated through *peer assessment*, several fine-tuning experiments were run using different combinations of models and

preference datasets.

The main focus of the experiments is the evaluation of the impact of the different preference dataset construction strategies on the alignment process. The use of two different models also makes it possible to verify whether the observed trends are consistent across different model architectures and sizes.

The experiments were conducted mainly on the Qwen 2.5 0.5B Instruct model, chosen for its small size and for the possibility of running numerous training configurations within the available computational limits. In addition, part of the experiments was replicated on the Gemma 2 2B Instruct model in order to verify whether the observed trends were also consistent on a larger model.

The datasets used all derive from the same AutoBench corpus described in Chapter 3, but differ in the preference construction strategy and in the filtering criteria.

In particular, the three preference datasets described in Section 3.5 were used:

- the Best-vs-Worst dataset;
- the Pairwise dataset obtained with threshold $\Delta = 0.3$ and a maximum of 20 pairs per prompt;
- the Pairwise dataset obtained with threshold $\Delta = 0.5$ and a maximum of 10 pairs per prompt.

Table 4.3 summarizes the experiments considered in this work.

Table 4.3. Summary of the experiments carried out.

Experiment	Model	Dataset
E1	Gemma 2 2B Instruct	Best vs Worst
E2	Qwen 2.5 0.5B Instruct	Best vs Worst
E3	Qwen 2.5 0.5B Instruct	Pairwise ($\Delta \geq 0.3$)
E4	Qwen 2.5 0.5B Instruct	Pairwise ($\Delta \geq 0.5$)

4.5 Evaluation Methodology

The evaluation of the fine-tuned models was carried out using two complementary approaches. On the one hand, performance on the AutoBench-derived test set was analyzed, in order to verify the model's ability to reproduce the preferences present in the training data; on the other hand, external public benchmarks were used, with the aim of measuring the possible transfer of the learned knowledge to different tasks.

This distinction makes it possible to assess separately the model's ability to learn the preference signal contained in the data and its possible ability to generalize to tasks not directly observed during the fine-tuning process.

4.5.1 Evaluation on the AutoBench Test Set

For each preference dataset, a separate test set was built through a split performed at the prompt level, as described in Chapter 3. This procedure ensures that no prompt present in the training set also appears in the test set, avoiding possible leakage.

Each element of the test set is a triplet in the format:

`prompt, chosen, rejected`

where `chosen` is the response preferred according to the AutoBench evaluation process and `rejected` is the response considered inferior.

The main metric used is preference accuracy, defined as the percentage of `(chosen, rejected)` pairs for which the model assigns a higher probability to the preferred response than to the rejected one.

Formally, given a prompt x and a pair of responses (y^+, y^-), the prediction is considered correct if:

$$\log \pi(y^+|x) > \log \pi(y^-|x)$$

where π denotes the evaluated model.

The accuracy obtained on the test set therefore provides a direct measure of the model's ability to learn the preferences extracted from the *peer assessment* process.

It is important to note that each preference dataset comes with its own test set, obtained through a split performed independently at the prompt level. Consequently, experiments conducted on different datasets are evaluated on different test sets. For this reason, the absolute accuracy values obtained by the base model are not directly comparable across different experiments; the correct comparison instead concerns the change between the base model and the corresponding fine-tuned version within the same experiment.

4.5.2 Raw Accuracy and Normalized Accuracy

Since the overall probability assigned to a sequence can be affected by its length, two different variants of the accuracy metric were considered.

The **raw accuracy** directly compares the total probability assigned to the two candidate responses.

The **normalized accuracy** instead uses the mean probability per token, reducing the advantage that could result from particularly short or particularly long responses.

The combined use of the two metrics makes it possible to distinguish genuine improvements in the quality of the learned preferences from possible effects due solely to the different length of the generated responses.

For each experiment, both measures are reported when available, allowing a more accurate analysis of the impact of fine-tuning.

4.5.3 External Benchmarks

Although the accuracy on the AutoBench test set is an important measure of preference learning, it does not make it possible to assess whether the acquired knowledge translates into improvements on different tasks.

For this reason, the models were subjected to a second evaluation phase using public benchmarks commonly employed in the literature on Large Language Models.

The benchmarks considered include:

- **ARC-Challenge** [13], consisting of multiple-choice science questions designed to assess reasoning and comprehension abilities;
- **HellaSwag** [14], focused on the completion of situations described in natural language and on commonsense understanding;
- **Winogrande** [15], used to analyze context-based reasoning and the resolution of linguistic ambiguities;
- **TruthfulQA** [16], designed to assess the model's ability to avoid false but plausible statements;
- **MMLU (Massive Multitask Language Understanding)** [17], which measures the model's general knowledge across numerous academic disciplines;
- **IFEval** [18], dedicated to assessing the ability to follow complex instructions.

These benchmarks make it possible to verify whether fine-tuning based on automatically generated data also produces observable effects outside the specific domain from which the preferences were extracted.

4.5.4 Evaluation Framework

The benchmarks were run using the *LM Evaluation Harness* framework [19], widely used by the research community for the standardized evaluation of language models.

For each model, performance before and after fine-tuning was compared, keeping the evaluation configuration unchanged. In this way, any variations in the observed metrics could be attributed solely to the effects of the alignment process.

For each experimental configuration, the following are reported:

- the performance of the base model;
- the performance of the model after DPO fine-tuning;
- the change observed between the two configurations.

The joint analysis of the results obtained on the AutoBench test set and on the external benchmarks therefore makes it possible to assess both preference learning and the generalization ability of the models trained with *peer-generated training data*.

Chapter 5

Experimental Results

5.1 Introduction to the Results

This chapter presents the results obtained from the fine-tuning experiments described in Chapter 4.

The goal of the analysis is to evaluate the effectiveness of the preference datasets built from the data generated by the AutoBench framework, verifying to what extent these data make it possible to align language models via Direct Preference Optimization.

For each experiment, the performance of the base model and of the corresponding fine-tuned version are compared. The evaluation is carried out both on the AutoBench-derived test set, used to measure the model's ability to learn the preferences present in the training data, and on external public benchmarks, used to analyze the possible generalization of the learned knowledge.

The experiments are organized according to the model used and the strategy adopted to build the preference dataset. In particular, the results obtained with the following are presented:

- the Best-Worst dataset, which associates with each prompt only the best and the worst available response;
- the Pairwise dataset with a minimum preference threshold of $\Delta = 0.3$;
- the Pairwise dataset with a minimum preference threshold of $\Delta = 0.5$;

The analysis of the results will highlight the role of the quality of the preference pairs and the impact of the different dataset construction strategies on the final performance of the models.

5.2 Gemma 2 2B + Best-Worst Dataset

The first experiment involved the *Gemma 2 2B Instruct* model, fine-tuned via Direct Preference Optimization using the preference dataset built with the Best-Worst strategy described in Chapter 3.

The dataset contains a single preference pair for each prompt, obtained by selecting the response with the highest score and the one with the lowest score among those generated during the *peer assessment* process. After the filtering and data preparation operations, the resulting dataset was split into a training set and an evaluation set, yielding 253 examples for training and 29 examples for internal evaluation during training.

Fine-tuning was carried out using the configuration described in Chapter 4, keeping all the main hyperparameters unchanged and training only the LoRA adapters via QLoRA.

5.2.1 Results on the AutoBench Test Set

Table 5.1 reports the accuracy obtained on the AutoBench-derived test set before and after fine-tuning.

Table 5.1. Accuracy on the AutoBench test set for Gemma 2 2B with the Best-Worst dataset.

Model	Accuracy (%)	Normalized accuracy (%)
Gemma 2 2B Base	54.84	67.74
Gemma 2 2B DPO	54.84	67.74

The results show that fine-tuning produced no measurable change in the model's ability to distinguish between preferred and non-preferred responses within the test set. Both the standard accuracy and the version normalized by response length remain unchanged with respect to the starting model.

5.2.2 Results on the External Benchmarks

To evaluate the possible transfer of the learned knowledge to different tasks, the model was also subjected to the benchmarks described in Chapter 4.

Table 5.2 summarizes the results obtained before and after fine-tuning.

Table 5.2. Performance of Gemma 2 2B on the external benchmarks.

Benchmark	Base (%)	DPO (%)	Δ
ARC-Challenge	49.20	48.80	-0.40
HellaSwag	66.00	64.40	-1.60
Winogrande	70.80	71.20	+0.40
TruthfulQA MC2	55.06	55.08	+0.02
MMLU	57.63	57.67	+0.04
IFEval	39.60	44.00	+4.40

The results show that fine-tuning produces generally small changes in the model's performance, confirming substantial stability with respect to the starting version.

The observed differences are below two percentage points for most benchmarks. In particular, Winogrande shows a slight increase of 0.4 percentage points, while MMLU and TruthfulQA MC2 remain essentially unchanged.

The most evident increase concerns IFEval, which rises from 39.6% to 44.0%, corresponding to an improvement of 4.4 percentage points. This result suggests a possible positive effect of fine-tuning on the model's ability to correctly follow instructions, although further experiments would be needed to verify its significance.

Slight decreases are instead observed on ARC-Challenge (-0.4 percentage points) and HellaSwag (-1.6 percentage points), both of limited magnitude.

Overall, fine-tuning on the Best-Worst dataset leaves the model's general capabilities nearly unchanged, while showing a significant improvement on IFEval and substantial stability on the other benchmarks considered.

5.3 Qwen 2.5 0.5B + Best-Worst Dataset

The second experiment involved the *Qwen 2.5 0.5B Instruct* model, fine-tuned via Direct Preference Optimization using the same Best-Worst dataset employed in the previous experiment.

Unlike Gemma 2 2B, Qwen 2.5 0.5B is a significantly more compact model, containing about 500 million parameters. This characteristic makes it particularly interesting for evaluating the effectiveness of the proposed approach in scenarios with limited computational resources.

Fine-tuning was carried out using the configuration described in Chapter 4 and the same Best-Worst dataset used in the previous experiment. This choice makes it possible to directly compare the behavior of Gemma 2 2B and Qwen 2.5 0.5B given the same training data, highlighting the influence of the architectural characteristics of the two models on the results obtained.

5.3.1 Results on the AutoBench Test Set

Table 5.3 reports the performance obtained on the AutoBench test set before and after fine-tuning.

Table 5.3. Accuracy on the AutoBench test set for Qwen 2.5 0.5B with the Best-Worst dataset.

Model	Accuracy (%)	Normalized accuracy (%)
Qwen 2.5 0.5B Base	29.03	38.71
Qwen 2.5 0.5B DPO	29.03	38.71

In this case too, fine-tuning produces no observable change in the accuracy on the test set. The model retains exactly the same performance in both the standard metric and the normalized version.

One noteworthy aspect concerns the absolute value of the accuracy obtained. Both the base model and the fine-tuned version in fact show performance below 50%, indicating a poor ability to reproduce the preferences present in the dataset. This behavior will be discussed in greater depth in Section 6.4.

5.3.2 Results on the External Benchmarks

Table 5.4 reports the results obtained on the external benchmarks.

Table 5.4. Performance of Qwen 2.5 0.5B on the external benchmarks.

Benchmark	Base (%)	DPO (%)	Δ
ARC-Challenge	36.40	36.40	0.00
HellaSwag	48.00	47.20	-0.80
Winogrande	59.20	58.40	-0.80
TruthfulQA MC2	42.37	42.31	-0.06
MMLU	45.92	46.11	+0.19
IFEval	20.00	16.40	-3.60

The results show generally small variations. MMLU records a slight increase of 0.19 percentage points, while ARC-Challenge remains unchanged.

Conversely, small decreases are observed on HellaSwag, Winogrande and TruthfulQA. The most evident change concerns IFEval, which drops from 20.0% to 16.4%, a reduction of 3.6 percentage points.

Overall, fine-tuning on the Best-Worst dataset does not lead to substantial improvements in the model's performance. The variations observed on the benchmarks are in fact limited and do not show a clear trend toward an increase in the model's general capabilities.

The absence of significant improvements suggests that, under the experimental conditions considered, the Best-Worst dataset provided a relatively weak learning signal for the Qwen 2.5 0.5B model. The possible causes of this behavior will be discussed in greater depth in Section 6.4.

5.4 Qwen 2.5 0.5B + Pairwise Dataset ($\Delta = 0.3$)

To evaluate the impact of the preference extraction procedure described in Chapter 3, a further experiment was conducted using the Qwen 2.5 0.5B Instruct model and a dataset built with the Pairwise strategy with threshold $\Delta = 0.3$.

Unlike the Best-Worst approach, which selects only the response with the highest score and the one with the lowest score for each prompt, the Pairwise method generates a preference pair whenever the difference between the scores of two responses exceeds a given threshold. In this way it is possible to obtain a larger number of training examples, while maintaining a minimum level of separation between the preferred and the rejected response.

Fine-tuning was carried out using the same DPO configuration described in Chapter 4.

5.4.1 Results on the AutoBench Test Set

Table 5.5 reports the performance obtained on the AutoBench test set.

Unlike what was observed with the Best-Worst dataset, in this case a slight increase in the raw accuracy is recorded, rising from 32.55% to 32.72%.

The normalized accuracy instead remains unchanged at 40.94%, suggesting that the improvement observed in the standard metric is extremely small.

Table 5.5. Accuracy on the AutoBench test set for Qwen 2.5 0.5B with the Pairwise dataset ($\Delta = 0.3$).

Model	Accuracy (%)	Normalized accuracy (%)
Qwen 2.5 0.5B Base	32.55	40.94
Qwen 2.5 0.5B DPO	32.72	40.94

Despite this increase, the overall performance remains well below the 50% threshold, indicating that the model continues to struggle to reproduce the preferences present in the dataset.

5.4.2 Results on the External Benchmarks

Table 5.6 reports the results obtained on the external benchmarks.

Table 5.6. Performance of Qwen 2.5 0.5B on the external benchmarks after fine-tuning with the Pairwise dataset ($\Delta = 0.3$).

Benchmark	Base (%)	DPO (%)	Δ
ARC-Challenge	36.40	36.40	0.00
HellaSwag	48.00	47.20	-0.80
Winogrande	59.20	59.20	0.00
TruthfulQA MC2	42.37	42.47	+0.10
MMLU	45.92	46.24	+0.32
IFEval	20.00	18.40	-1.60

The results show generally modest variations, similarly to what was observed in the Best-Worst experiment.

The most evident increase concerns MMLU, which records an improvement of 0.32 percentage points. TruthfulQA MC2 also shows a slight gain (+0.10 points), while ARC-Challenge and Winogrande remain unchanged.

The most relevant decreases affect HellaSwag (-0.80 points) and IFEval (-1.60 points), although the latter is less marked than what was observed with the Best-Worst dataset.

Overall, the Pairwise dataset ($\Delta = 0.3$) appears to produce slightly better results than the Best-Worst approach, both in terms of accuracy on the test set and for some external benchmarks. However, the improvements observed remain limited and do not point to a substantial increase in the model's capabilities.

5.5 Qwen 2.5 0.5B + Pairwise Dataset ($\Delta = 0.5$)

The last experiment considered in this work again uses the *Qwen 2.5 0.5B Instruct* model, trained via Direct Preference Optimization on a dataset built with the Pairwise strategy with threshold $\Delta = 0.5$.

As discussed in Chapter 3, raising the minimum required threshold between the scores of the two responses results in a more restrictive selection of preference

pairs. In theory, this choice yields examples characterized by clearer preferences and therefore potentially more informative for the alignment process.

The goal of the experiment is to assess whether a more rigorous selection of preference pairs produces different effects compared to the Pairwise configuration with threshold $\Delta = 0.3$.

5.5.1 Results on the AutoBench Test Set

To analyze the evolution of the model during the training process, fine-tuning was run for a maximum of five epochs, saving a checkpoint at the end of each one. The accuracy was then evaluated on the intermediate checkpoints to verify whether progressive improvements occurred.

Table 5.7 reports the results obtained for the base model and for the checkpoints of the first and second epochs. The later checkpoints are not reported because they showed no appreciable changes with respect to the second epoch.

Table 5.7. Accuracy on the AutoBench test set for Qwen 2.5 0.5B with the Pairwise dataset ($\Delta = 0.5$).

Checkpoint	Accuracy (%)	Normalized accuracy (%)
Base	29.55	41.56
1 epoch	29.55	41.88
2 epochs	29.87	42.52

The performance on the test set shows a slight improvement already in the early stages of training. After the first epoch the raw accuracy remains unchanged with respect to the base model, while the normalized accuracy increases from 41.56% to 41.88%. At the end of the second epoch, a further increase in both metrics is observed, with the accuracy reaching 29.87% and the normalized accuracy 42.52%. The later checkpoints show no significant variations with respect to these values, suggesting that the optimization process quickly reaches convergence.

5.5.2 Results on the External Benchmarks

Table 5.8 reports the performance obtained on the public benchmarks.

Table 5.8. Performance of Qwen 2.5 0.5B on the external benchmarks during fine-tuning with Pairwise ($\Delta = 0.5$).

Benchmark	Base	1 epoch	2 epochs
ARC-Challenge	36.40	32.00	33.00
HellaSwag	48.00	49.00	49.00
Winogrande	59.20	58.00	58.00
TruthfulQA MC2	42.37	39.76	40.02
MMLU	45.92	45.84	46.07
IFEval	20.00	15.00	16.00

The results on the external benchmarks show small but non-uniform variations during the training process. In particular, ARC-Challenge records a decrease with

respect to the base model, going from 36.40% to 32.00% after the first epoch and partially recovering to 33.00% in the second. HellaSwag is the only benchmark that shows a slight, stable improvement, reaching 49.00% already after the first epoch and keeping this value in the second as well. Winogrande instead undergoes a reduction of about one percentage point, with no further changes between checkpoints.

TruthfulQA MC2 and IFEval also show a decrease with respect to the starting model, with a partial recovery in the second epoch that nonetheless does not bring them back to the initial performance. MMLU remains substantially stable throughout training, showing only a slight final increase, from 45.92% to 46.07%.

Overall, fine-tuning with the Pairwise dataset ($\Delta = 0.5$) produces limited effects on the external benchmarks as well. Similarly to what was observed on the AutoBench test set, most of the changes already appear in the first two epochs, while the later checkpoints show no significant improvements, suggesting a rapid convergence of the optimization process.

5.6 Summary of the Results

The experiments presented in the previous sections show that fine-tuning via Direct Preference Optimization on automatically generated preference datasets has generally limited effects on the performance of the models considered.

In the case of Gemma 2 2B, fine-tuning leaves the performance on the AutoBench test set substantially unchanged and produces only slight variations on the external benchmarks. Similarly, the Qwen 2.5 0.5B model trained on the Best-Worst dataset shows no significant improvements with respect to the starting version.

The comparison between the different extraction strategies highlights a noteworthy pattern: raising the threshold Δ in the Pairwise strategy produces a clearer improvement in the preference accuracy on the internal test set, while the effect on the external benchmarks remains limited and non-uniform. This aspect is explored further in Section 6.2.

Overall, the experiments show that *peer-generated training data* are a usable source for fine-tuning via DPO, although, in the configurations considered, they do not produce substantial increases in the models' performance.

The possible causes of these results, the methodological implications that emerged during experimentation and the limitations of the proposed approach are discussed in the next chapter.

Chapter 6

Discussion of the Results

6.1 Assessing the Effectiveness of Peer-Generated Training Data

The main objective of this work was to assess the possibility of using preference data automatically generated by other Large Language Models as a source of supervision for fine-tuning via Direct Preference Optimization, without resorting to human annotations.

Overall, the results obtained show that this approach is technically feasible. It was indeed possible to build preference datasets from the scores produced by the AutoBench framework, use them to fine-tune open-source language models and systematically evaluate their effects both on a test set derived from the same framework and on standardized public benchmarks.

In terms of performance, the experiments show no marked improvements over the starting models. In most of the configurations considered, fine-tuning produces small variations in both the preference accuracy and the external benchmarks, indicating that the use of relatively small, automatically built datasets is not, by itself, sufficient to substantially change the model's behavior.

However, the absence of significant improvements does not necessarily imply that the *peer-generated training data* paradigm is ineffective. On the contrary, some experiments show that datasets characterized by more reliable preferences produce slightly better effects than those obtained with simpler extraction strategies. This suggests that the quality of the preference signal is a determining factor for the success of the alignment process.

A further element that emerged from the experiments is the substantial stability of the models' performance on the external benchmarks. In most cases, fine-tuning does not cause a significant degradation of the model's general capabilities, indicating that updating via DPO, at least in the configurations considered, tends to modify the model's behavior in a relatively conservative manner.

Overall, the results obtained therefore indicate that automatically generated preference data are a possible source of supervision for the alignment of Large Language Models. At the same time, they show that the effectiveness of this approach depends to a large extent on the quality of the preferences used, suggesting that dataset construction is as important an aspect as the choice of the optimization

algorithm employed for fine-tuning.

6.2 Influence of Preference Dataset Quality

One of the aspects that emerged most clearly during the experimental campaign is the importance of the quality of the preference dataset used for fine-tuning. Even when adopting the same training procedure, the same hyperparameters and the same starting models, the results obtained show differences attributable mainly to the characteristics of the dataset employed.

The experiments conducted in this work made it possible to compare different strategies for building preference pairs. In particular, we considered a dataset obtained through Best-Worst selection and two Pairwise datasets built by imposing different minimum thresholds on the score gap between responses ($\Delta = 0.3$ and $\Delta = 0.5$). All datasets were generated following the same processing pipeline described in Chapter 3 and used with the experimental configuration unchanged, thus allowing a direct comparison between the different preference extraction strategies.

The results show that the Best-Worst dataset and the Pairwise dataset with threshold $\Delta = 0.3$ produce generally limited effects on the models' performance. The variations observed in both the preference accuracy and the external benchmarks are in fact small and, in most cases, within a few percentage points.

The comparison with the Pairwise dataset built using a higher threshold ($\Delta = 0.5$) instead shows slightly more marked improvements on some evaluation metrics. Although these increases remain modest, they suggest that a more restrictive selection of preference pairs may provide a more informative learning signal than comparisons between responses of very similar quality.

A possible explanation lies in the nature of the pairs used during fine-tuning. When the quality gap between two responses is small, both may be plausible or may differ only in marginal respects. In these situations the model receives a relatively weak preference signal, which makes it harder to learn a behavior significantly different from the initial one. Conversely, pairs characterized by more evident quality differences provide clearer indications about which response to prefer, facilitating the optimization process.

The results obtained therefore suggest that the mere availability of a large number of examples is not sufficient to guarantee effective alignment. The quality of the preferences used during fine-tuning in fact appears to play a role at least as important as the quantity of available data.

This observation is particularly relevant in the context of *peer-generated training data*. Unlike preferences collected from human annotators, the judgments automatically produced by other language models may suffer from evaluation errors, disagreements among the models involved or noise introduced during the *peer assessment* process. Consequently, the dataset construction phase takes on central importance: procedures for filtering, selecting and validating preference pairs can significantly affect the quality of the signal used during fine-tuning.

Overall, the experiments conducted in this work therefore suggest that the main limiting factor is not the DPO algorithm, which proved stable in all the configurations considered, but rather the quality of the information provided during

training. Improving the strategies for building preference datasets thus appears to be one of the most promising directions for increasing the effectiveness of alignment methods based on automatically generated data.

6.3 Critical Interpretation of the Experimental Results

The results presented in Chapter 5 show that fine-tuning via Direct Preference Optimization produces, in most experiments, small variations in both the accuracy on the AutoBench test set and the public benchmarks used for evaluation. Although these improvements may appear limited, they are consistent with the characteristics of the experimental configuration adopted in this work.

A first element to consider is the size of the training datasets. The preference datasets built from the AutoBench data contain a relatively small number of examples compared to the datasets commonly used for the alignment of Large Language Models. In particular, the Best-Worst dataset comprises about three hundred preference triplets, while the Pairwise datasets reach a few thousand examples. Although sufficient to study the effect of different preference construction strategies, these quantities inevitably limit the extent of the changes the model can learn during fine-tuning.

A second aspect concerns the very nature of Direct Preference Optimization. Unlike more aggressive training techniques, DPO is designed to modify the model's behavior solely on the basis of the observed preferences, while maintaining a strong proximity to the distribution learned during pre-training. This conservative behavior is one of the algorithm's main strengths, as it reduces the risk of degrading the model's general capabilities, but it also leads to generally gradual improvements, especially when the available preference signal is limited.

The choice to use *Parameter-Efficient Fine-Tuning* techniques also helps explain the observed results. In all experiments only the LoRA adapters were updated, keeping the parameters of the base model frozen. This approach drastically reduces the computational requirements and makes training on limited hardware possible, but inevitably offers a lower adaptation capacity than a full fine-tuning of the entire model.

Finally, the experiments were conducted on relatively small models, Gemma 2 2B Instruct and Qwen 2.5 0.5B Instruct. Although these models are an interesting choice for assessing the feasibility of the approach in accessible settings, they have a lower representational capacity than larger Large Language Models, making it plausible that the effect of fine-tuning is less marked.

In light of these considerations, the results obtained appear consistent with the experimental context of this work. In particular, the fact that fine-tuning produced no substantial variations on the external benchmarks, while generally keeping the model's performance stable, suggests that the use of automatically generated preference data is a promising approach but one that requires sufficiently large and accurate datasets to produce more evident improvements. The results obtained therefore highlight the importance not only of the availability of preference data, but also of their quality and their ability to provide a sufficiently strong informative signal during the alignment process.

6.4 Biases Observed During Evaluation

6.4.1 Length Bias

During the evaluation of the models, a methodological issue emerged that significantly affected the interpretation of the results obtained on the AutoBench test set. In particular, during the first experiments it was observed that the Qwen 2.5 0.5B model obtained a preference accuracy systematically below 50%, both before and after fine-tuning.

A result of this kind was hard to justify. The preference accuracy is in fact computed by comparing, for each prompt, the probability assigned by the model to the *chosen* response and to the *rejected* one. A consistently below-50% value would suggest that the model systematically prefers the response deemed worse by the dataset, achieving performance lower than that expected from random choice.

To understand the origin of this behavior, the way in which the log-probabilities of the two responses were compared was analyzed. In the initial formulation of the accuracy, the sum of the log-probabilities of the tokens making up each response was used. Since this quantity grows in absolute value as the sequence length increases, longer responses tend naturally to accumulate more negative log-probabilities than shorter ones, regardless of their quality.

To reduce this effect, a second metric was introduced, obtained by normalizing the overall log-probability by the number of tokens in the response. In this way the comparison is made using the mean log-probability per token, limiting the influence of the different lengths of the generated sequences.

The introduction of this normalization produced an increase in the accuracy observed for both models. In the case of Gemma 2 2B the accuracy rose from 54.84% to 67.74%, while for Qwen 2.5 0.5B the values observed ranged between about 39% and 42%, depending on the evaluation dataset used. Although the latter result remains below 50%, it nonetheless shows that a significant part of the performance initially observed was attributable to the bias introduced by response length.

It is important to stress that the normalization changes neither the behavior of the model nor the performance obtained on the external benchmarks, but only affects the way in which the preference accuracy is evaluated. It is therefore a measure aimed at making the comparison between responses of different length fairer.

The observation of this phenomenon also suggests a more general reflection on the evaluation of models trained on preference datasets. When the compared responses differ significantly in length, using the total log-probability directly can introduce a systematic bias in the evaluation metric. Although length normalization does not completely eliminate this problem, it is a simple and effective measure to reduce its impact and obtain a measure more representative of the preferences actually learned by the model.

6.4.2 An Additional Hypothesis: Typicality Bias

Length normalization corrects much of the distortion introduced by comparing responses of different lengths, but it is not sufficient to fully explain the behavior observed for Qwen 2.5 0.5B, whose normalized preference accuracy remains consistently below the 50% threshold (38.71%–42.52% depending on the dataset), compared to

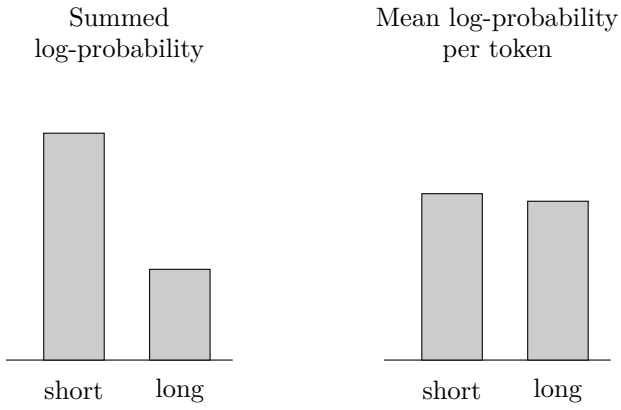


Figure 6.1. Qualitative illustration of the effect of normalization: with the summed log-probability (left) the longer response is penalized far more heavily, regardless of its quality; when normalizing by the number of tokens (right) the two responses become comparable.

the 67.74% obtained by Gemma 2 2B. If length bias were the only factor at play, one would expect normalization to bring the accuracy back at least close to the level of random choice; the residual gap suggests the presence of a second factor.

A possible explanation finds support in a line of research that studies the relationship between the probability a language model assigns to a sequence and its perceived quality. Meister et al. [20] observe that the strings of maximum probability under a language model generally do not coincide with those judged best by a human reader, tending instead to be generic and predictable. Tan et al. [21] extend the observation to models aligned via RLHF, showing that, beyond a certain threshold, a further increase in the probability assigned to a response is associated with a decrease in perceived quality. Although this result concerns RLHF, the same dynamic appears plausible for DPO as well, which shares with RLHF the objective of maximizing, under a KL regularization constraint, the probability of the preferred responses.

Applied to the case discussed here, this relationship provides a possible interpretive key: the *chosen* responses in the AutoBench dataset are selected through the collective judgment of a heterogeneous pool of *peer* models, a process that tends to reward responses that are more original or distinctive with respect to the average of the generators involved, while the *rejected* responses may be more generic. If the chosen responses are, in aggregate, less "typical" under the distribution of a small model, that model will tend to assign them a lower probability even before any fine-tuning – an effect consistent with the *likelihood bias* phenomenon also documented in automatic evaluation via LLM-as-a-Judge by Oi et al. [22].

On the basis of the collected data alone, it is not possible to establish with certainty whether and how much model scale modulates this effect: the comparison between Gemma 2 2B and Qwen 2.5 0.5B involves too large a difference in parameters, and differences in the pre-training corpus or in tokenization could contribute equally. The explanation presented here therefore remains a plausible interpretive hypothesis consistent with the literature, not a direct demonstration: a verification of it – for example by comparing the stylistic typicality of the *chosen* and *rejected* responses,

or by repeating the experiment on models of intermediate scale – is proposed as a direction for future research in Section 6.6.

What the observation suggests with greater certainty is a more general methodological limitation: preference accuracy computed via log-probabilities, even once normalized by length, is not a neutral metric with respect to the stylistic typicality of the compared responses, especially when the evaluated model has limited representational capacity relative to the pool of models that generated the reference preferences. This does not, however, undermine the relative comparison between the base model and the fine-tuned model within the same experiment, which remains the most reliable measure of the effect of training.

6.5 Limitations of This Work

Although the experiments conducted made it possible to analyze the behavior of fine-tuning using automatically generated preference data, this work has some limitations that must be considered when interpreting the results.

A first limitation concerns the number of models evaluated. The experiments were in fact conducted exclusively on Gemma 2 2B Instruct and Qwen 2.5 0.5B Instruct, both belonging to the category of relatively small Large Language Models. Although this choice made it possible to run numerous experiments on accessible hardware, the conclusions obtained cannot automatically be extended to larger models or to models belonging to different families.

A second limitation concerns the number and variety of the preference datasets considered. In this work, different strategies for building preference pairs were compared, including Best-Worst and Pairwise with different thresholds, but all derive from the same data source, namely AutoBench. It would therefore be interesting to verify whether the same observations hold when using datasets obtained through different automatic preference generation procedures.

From a computational standpoint, the experiments were run using a single NVIDIA Tesla T4 GPU with 14.6 GB of video memory. Although the use of QLoRA and 4-bit quantization made it possible to fine-tune the selected models, this configuration inevitably limited the size of the models that could be used, the number of experiments that could be run and the possibility of exploring more complex training configurations.

This same memory constraint made it necessary to limit the maximum length of the sequences tokenized during training (Section 4.3.1), set to 850 tokens for Gemma 2 2B Instruct and 2000 tokens for Qwen 2.5 0.5B Instruct, with a stricter limit for the former due to its larger memory footprint. Since the responses produced during the AutoBench *peer assessment* process are often quite long, these thresholds may have led to the truncation of part of the content of the longer responses, particularly in the case of Gemma 2 2B. Truncation of this kind risks altering the preference signal observed by the model during training, for instance by cutting off portions of a response relevant to the comparison between `chosen` and `rejected`, and thus constitutes a further factor, of a purely infrastructural nature, that may have contributed to limiting the effectiveness of fine-tuning observed in the experiments conducted on this model.

A further limitation is the absence of a direct comparison with preference datasets annotated by humans. The goal of this work was in fact not to compare the *peer-generated training data* paradigm with traditional Reinforcement Learning from Human Feedback, but to assess the feasibility of using preferences generated exclusively by other language models. Consequently, the results obtained do not make it possible to establish how closely automatic preferences approach, in terms of effectiveness, those produced by human annotators.

Finally, the evaluation was conducted using a limited set of public benchmarks and through the accuracy on the AutoBench-derived test set. Although these metrics are well-established tools for evaluating Large Language Models, they do not cover all aspects of a language model's behavior, such as robustness, generalization ability on specialized domains or the quality perceived by users during real interactions.

Overall, these limitations do not diminish the interest of the results presented, but outline more precisely the context within which they should be interpreted. This work is in fact intended as an exploratory study aimed at assessing the feasibility of using automatically generated preference data for the alignment of Large Language Models, providing an experimental basis on which to develop broader and more in-depth analyses in the future.

6.6 Future Directions

The results obtained in this work point to several possible research directions that could help further clarify the potential of *peer-generated training data* for the alignment of Large Language Models.

A first direction concerns the use of larger preference datasets. The experiments have shown that the quality of the dataset significantly influences the final performance of the model; however, the number of available examples remains relatively small by current LLM alignment standards. Applying the proposed pipeline to larger versions of AutoBench could make it possible to assess whether the observed effects become more evident as the amount of available data increases.

A second development consists in extending the experimentation to larger language models. Although Gemma 2 2B and Qwen 2.5 0.5B were an adequate choice for the goals of this work, models with a larger number of parameters might benefit to a different extent from fine-tuning with automatically generated preferences. A systematic comparison between models from different families and scales would make it possible to assess how generalizable the conclusions obtained are.

A further area of research concerns improving the quality of automatic preferences. In this work the datasets were built from the scores produced by the AutoBench *peer assessment* process, but more sophisticated strategies for selecting preference pairs could be investigated, for example by combining the judgments of different models, introducing additional filtering criteria or assigning a different weight to preferences according to the level of agreement among the evaluator models.

The observations that emerged during the evaluation also suggest the opportunity to investigate in greater depth the methodological aspects related to the construction and evaluation of preference datasets. In particular, the length bias observed in the computation of the accuracy shows how the choice of metric can significantly

affect the interpretation of the results. The development of more robust metrics or alternative evaluation procedures could help obtain more reliable estimates of the models' ability to learn preferences.

Finally, a natural extension of this work is to directly compare automatically generated preference datasets with datasets built through human annotation. An analysis of this kind would make it possible to assess not only the differences in the final performance of the models, but also the trade-off between the quality of the preference signal, data collection costs and the scalability of the alignment process.

Overall, this work represents a first step in the analysis of the use of automatically generated preference data for the fine-tuning of Large Language Models. Although the results obtained show modest improvements, they confirm the feasibility of the approach and suggest that the quality of the preference data is a determining factor for the effectiveness of alignment. The evolution of automatic evaluation frameworks and the growing availability of increasingly advanced language models make it plausible that *peer-generated training data* may take on an increasingly relevant role in the development of future alignment techniques.

Chapter 7

Conclusions

This work has analyzed the possibility of using preference data generated automatically through *peer assessment* processes among Large Language Models as a data source for fine-tuning via *Direct Preference Optimization* (DPO), without resorting to human annotations.

Starting from the data produced by the AutoBench framework, a complete pipeline was developed for building preference datasets in the `(prompt, chosen, rejected)` format. The pipeline comprises the stages of data preparation, filtering, deduplication and extraction of preference pairs according to different strategies, making it possible to transform the evaluations automatically produced by the models into data usable for aligning other language models.

The validity of the approach was assessed through an experimental campaign conducted on two open-source Large Language Models, Gemma 2 2B Instruct and Qwen 2.5 0.5B Instruct, using several preference datasets built from the same starting data but with different pair selection criteria. The performance of the models was analyzed both on a test set derived from AutoBench and on standardized public benchmarks, allowing preference learning and generalization ability to be assessed at the same time.

The results show that fine-tuning based on *peer-generated training data* is indeed feasible and does not cause a significant degradation of the models' general capabilities. At the same time, the experiments show that the improvements obtained are generally modest and depend to a considerable extent on the quality of the preference dataset used. In particular, selecting pairs characterized by larger score differences tends to produce a more effective learning signal than less selective strategies — an effect observed more clearly on the internal preference accuracy than on the external benchmarks — confirming that the quality of the preference signal is an element at least as central as the quantity of data for the effectiveness of the alignment process.

During the experimental work, a bias related to response length was also identified in the computation of log-probabilities, which affected the evaluation of preference accuracy, particularly in the case of the Qwen 2.5 0.5B model. Once this bias was corrected, a second factor emerged, related to the stylistic typicality of the responses with respect to the distribution learned by the model, which may help explain the residual gap between the two models considered; this is, however, an interpretive hypothesis that would require targeted verification to be confirmed. The analysis

of these phenomena nonetheless highlighted the importance of using appropriate evaluation metrics and made it possible to correctly interpret results that, at first analysis, appeared hard to explain.

Although this work was conducted on a limited number of models and datasets, the results obtained provide interesting indications of the potential of automatically generated preference data. In particular, it emerges that the main limiting factor is not the absence of human annotations, but rather the quality of the extracted preferences and the ability of the evaluation process to produce sufficiently reliable and informative comparisons.

In conclusion, this work confirms the feasibility of using *peer-generated training data* as a possible alternative, at least in some contexts, to traditional human preference datasets for the fine-tuning of Large Language Models. While not a complete substitute for human annotations, this approach offers interesting prospects in terms of scalability, data collection costs and automation of the alignment process. The evolution of automatic evaluation frameworks and the improvement of the models used as judges could make this paradigm an increasingly relevant component in the development of future alignment systems for Large Language Models.

Bibliography

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[3] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[4] Daniel Han, Michael Han, and Unsloth team. Unsloth, 2023. https://github.com/unslothai/unsloth.

[5] Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.

[6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[7] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[8] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[9] Ruosen Li, Teerth Patel, and Xinya Du. Prd: Peer rank and discussion improve large language model based evaluations. *Transactions on Machine Learning Research*, 2024.

[10] Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Bingxiang He, Wei Zhu, Yuan Ni, Guotong Xie, Ruobing Xie, Yankai Lin, Zhiyuan Liu, and Maosong Sun. ULTRAFEEDBACK: Boosting language models with scaled AI feedback. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 9722–9744. PMLR, 2024.

[11] Dario Loi, Elena Maria Muià, Federico Siciliano, Giovanni Trappolini, Vincenzo Crisà, Peter Kruger, and Fabrizio Silvestri. Autobench: Automating llm evaluation through reciprocal peer assessment. *arXiv preprint arXiv:2510.22593*, 2025.

[12] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformer reinforcement learning, 2020. https://github.com/huggingface/trl.

[13] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

[14] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really predict the next event? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800, 2019.

[15] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.

[16] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3214–3252, 2022.

[17] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.

[18] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.

[19] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, 12 2023.

[20] Clara Meister, Tiago Pimentel, Gian Wiher, and Ryan Cotterell. Locally typical sampling. *Transactions of the Association for Computational Linguistics*, 11:102–121, 2023.

[21] Naaman Tan, Josef Valvoda, Tianyu Liu, Anej Svete, Yanxia Qin, Min-Yen Kan, and Ryan Cotterell. A probability–quality trade-off in aligned language models and its relation to sampling adaptors. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 14805–14829, 2024.

[22] Masanari Oi, Masahiro Kaneko, Ryuto Koike, Mengsay Loem, and Naoaki Okazaki. Likelihood-based mitigation of evaluation bias in large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 3237–3245, 2024.